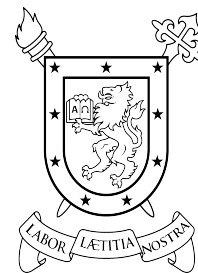


UNIVERSIDAD DE SANTIAGO DE CHILE
FACULTAD DE CIENCIA
Departamento de Matemática y Ciencia de la Computación



Implementación de un algoritmo de clasificación en línea usando técnicas de Ensemble learning

Yerko Carreño Pérez

Profesor guía: Felipe Elorrieta López

Trabajo de titulación presentado a la
Facultad de Ciencia en cumplimiento
parcial de los requisitos exigidos para
optar al título de Ingeniero Estadístico

Santiago, Chile

2020

© 2020, Yerko Carreño Pérez

Todos los derechos reservados. Queda prohibida la reproducción total o parcial sin autorización previa y por escrito.

DEDICATORIA

A mi mamá

Agradecimientos

Me gustaría agradecer a todas las personas que estuvieron presentes en esta etapa, la cual fue muy enriquecedora y en algunos momentos muy complicada.

Quiero dar gracias a mi familia, especialmente a mi madre, la cual siempre ha sido un apoyo incondicional en todas las circunstancias de mi vida, sin ella nunca podría haber entrado a estudiar a la educación superior, lo que ha sido un real privilegio del cual no todos los chilenos podemos acceder, gracias por siempre creer en mí. A mis amigos, compañeros y colegas que conocí en todos estos años de estudios y anécdotas, los cuales han sido un apoyo fundamental, en este proceso, aportando y ayudando a mejorar en todos los aspectos de formación, especialmente a Giovanni, Rodrigo, Felipe, Matías, Italo, Katherine y Carolina.

A mis profesores de carrera, los cuales me formaron tanto académica como moralmente todos estos años, fortaleciendo la perseverancia para superar el fracaso, en especial a mi profesor guía Felipe, el cual me apoyó y guió en el desarrollo de este trabajo, aportando con su experiencia y conocimiento digno de admirar.

También a mis compañeros de trabajo, los cuales me motivaron a emprender nuevos desafíos y creer en mí, dando un empujón importante para empezar a desarrollarme profesionalmente y ayudando a tener una experiencia laboral muy grata, especialmente a Felipe, Marcelo, Patricio, Germán y Tomás.

Gracias a todos, por sus influencias y buenas vibras, sin ustedes esto no sería posible.

Resumen

Debido a la actual necesidad del procesamiento masivo de información de forma rápida y online, es que es necesario que los algoritmos de machine learning tengan la capacidad de entrenarse con un enfoque de aprendizaje online o streaming. En respuesta a esto, se proponen los métodos de Ensemble Learning, los cuales son una técnica de aprendizaje automático que combina varios modelos básicos llamados Weak Learners o aprendiz débil, cuyo rendimiento es levemente mejor que aleatorio y los que son usados como modelos bases para producir un modelo óptimo. Para este fin, se evaluará la complejidad algorítmica de los principales Weak Learners para determinar cuál usar como base del modelo de Ensemble.

Bagging y Boosting son los principales algoritmos de Ensemble Learning, y son los que obtienen mejores resultados al momento de mejorar el rendimiento de un modelo, encontrando un equilibrio entre el sesgo y la varianza.

En este trabajo se implementarán los modelos de Ensemble Learning propuestos por Oza (2005) para entrenar modelos con un enfoque online, y se realizará una comparación con los modelos clásicos de enfoque de aprendizaje Batch. Los modelos se encuentran disponibles en los software de código abierto Python (Python Software Foundation, 1995) y R (R Core Team, 2014), en los packages Scikit-multiflow (Montiel et al., 2018) y RMOA (Wijffels, 2014), respectivamente.

Los modelos serán implementados a problemas de clasificación. El primer dataset corresponde a un conjunto de información de estrellas, donde se busca la detección de una estrella Pulsar, y el segundo dataset corresponde a información de alertas astronómicas recibidas por el telescopio ZTF (Bellm et al., 2019), el cual contiene información de alertas astronómicas categorizadas entre transientes, periódicos y estocásticos.

Palabras claves: Machine Learning, Ensemble Learning, Streaming data, Batch, Weak Learners, Bagging, Boosting.

Abstract

Given the current need of processing massive amounts of information in a quick and online manner, it is necessary that machine learning algorithms have the capacity to be trained with an online or streaming learning approach. In response to this, Ensemble Learning methods are proposed, an automatic learning technique that combines several basic models called Weak Learners, which performance is slightly better than chance and are used as base models to produce an optimal model. For this purpose, the algorithmic complexity of the main Weak Learners will be evaluated to determine which to use as a basis of the Ensemble model.

Bagging and Boosting are the main algorithms of Ensemble Learning and those that obtain the best results when improving the performance of a model, finding a balance between bias and variance.

In this work, the Ensemble Learning models proposed by Oza (2005) will be implemented to train models with an online approach, and a comparison will be made with the classic models of the Batch learning approach. The models are available in Python (Python Software Foundation, 1995) and R (R Core Team, 2014) open-source softwares, in Scikit-multiflow (Montiel et al., 2018) and RMOA (Wijffels, 2014) packages, respectively.

The models will be implemented to classification problems. The first dataset corresponds to a set of stars information, where the aim is the detection of a Pulsar star, and the second dataset corresponds to information from astronomical alerts received by the ZTF (Bellm et al., 2019) telescope, which contains information from astronomical events categorized among transients, periodic and stochastic.

Keywords: Machine Learning, Ensemble Learning, Streaming data, Batch, Weak Learners, Bagging, Boosting.

Índice de imágenes y gráficos

2.1. Perceptrón de una capa (SONG, 2018) [Figura]. Recuperado de https://tianyusong.com/	14
2.2. Función sigmoide elaboración propia	16
2.3. Muestreo bootstrapping (Rocca, 2019) [Figura].	19
2.4. Equilibrio Sesgo-Varianza (Rocca, 2019) [Figura]. Recuperado de https://towardsdatascience.com/	21
2.5. Batch Bagging (Oza, 2005)	22
2.6. Comparación entre un modelo base, Bagging y Boosting [Figura].	23
2.7. Online Bagging (Oza, 2005)	26
2.8. Comparación de tipos de complejidad [Figura]. Recuperado de https://notas.uristolar.com/complejidad-algoritmica	27
3.1. Comparación aproximación complejidad tiempo entre los Weak Learners	44
3.2. Bagging Online	47
3.3. Boosting Online	48
3.4. Comparación métricas simulados	48
5.1. Taxonomía definida en colaboración ALeRCE	66
5.2. Numero de modelos bases	69
5.3. Bagging Online	70

5.4. Boosting Online	71
--------------------------------	----

Índice de tablas

2.1. Matriz de confusión	29
3.1. Muestra de datos simulados	42
3.2. Acurracy Weak Learners simulados	43
3.3. Tiempos simulados	45
3.4. Ensembles simulados	46
3.5. Weak Learners simulados en R	52
3.6. Ensembles simulados en R	53
4.1. Resumen HTRU	56
4.2. Resumen descriptivo HTRU	57
4.3. Comparación métricas Weak Learners con enfoque Batch Pulsar	58
4.4. Comparación Bagging y Boosting con enfoque Batch Pulsar	58
4.5. Comparación Batch y Online Pulsar	59
5.1. Resumen ZTF	67
5.2. Comparación métricas Weak Learners con enfoque Batch ALerCE	69
5.3. Comparación Bagging y Boosting con enfoque Batch ALerCE	70
5.4. Comparación Batch y Online ALerCE	71

Índice general

1. Introducción	1
1.1. Motivación	1
1.2. Planteamiento del problema	3
1.3. Objetivos	6
1.3.1. Objetivo general	6
1.3.2. Objetivos específicos	6
2. Marco teórico	7
2.1. Machine Learning	7
2.2. Aprendizaje supervisado	8
2.2.1. Regresión logística	8
2.2.2. Árboles de decisión	11
2.2.3. Perceptrón Simple	13
2.2.4. Naives Bayes	16
2.2.5. Ensemble Learning	18
2.2.5.1. Preliminares:	18
2.2.5.2. Modelos de Ensemble Learning	20
2.2.5.3. Bagging: Bootstrap Aggregation:	21
2.2.5.4. Boosting	23
2.3. Online Ensemble Learning	24

2.3.1.	Tipos de Aprendizaje	24
2.3.1.1.	Aprendizaje Batch o Lote:	24
2.3.1.2.	Aprendizaje Online o Streaming:	24
2.3.2.	Bagging Online	25
2.3.3.	Complejidad Algorítmica	26
2.3.3.1.	Notación Big O	26
2.4.	Métricas de clasificación	29
2.4.1.	Matriz de confusión	29
2.4.1.1.	Accuracy	30
2.4.1.2.	Precisión	30
2.4.1.3.	Sensibilidad o Recall	31
2.4.1.4.	Especificidad	31
2.4.2.	Curva ROC (receiver operating characteristic curve)	31
2.4.3.	KS (Kolmogórov-Smirnov)	31
2.4.4.	Índice GINI	32
2.4.5.	Norma F1 Score	33
3.	Implementación software	34
3.1.	Python	34
3.1.1.	Scikit-learn	34
3.1.2.	Scikit-Multiflow (Multi-output Streaming Framework)	38
3.1.2.1.	FileStream	38
3.1.2.2.	Evaluation	39
3.1.2.3.	Ensemble methods	41
3.1.3.	Simulación	41
3.1.3.1.	Weak Learners	42
3.1.3.2.	Complejidad Algorítmica	43
3.1.3.3.	Ensemble Learning	45

3.1.3.4. Online Ensemble Learning	46
3.2. R Project	49
3.2.1. Caret	49
3.2.2. RMOA (Massive Online Analysis)	51
3.2.3. Simulación	52
4. Aplicación problema de clasificación binario: Estrella Pulsar	54
4.1. Información del conjunto de datos	54
4.2. Aplicación	56
4.2.1. Análisis Descriptivo	57
4.2.2. Modelos Batch	58
4.2.3. Modelos Online	58
5. Aplicación problema clasificación multiclase: Proyecto Alerce	60
5.1. Información del conjunto de datos	60
5.1.1. Zwicky Transient Facility (ZTF):	61
5.1.1.1. Variables de Detección	62
5.1.2. Modelos Batch:	68
5.1.3. Modelos Online:	70
6. Conclusiones	72
7. Anexos	74
Referencias	80

Capítulo 1

Introducción

1.1. Motivación

Estamos viviendo en la era de la información, el mundo y la tecnología están en constante evolución y cada vez está más ligada a nuestras vidas, es por esto y el explosivo aumento de datos de variadas fuentes de información, tanto de datos estructurados (filas y columnas), como datos no estructurados (imágenes, vídeos, etc...) surge la necesidad de almacenar estos datos de manera tal que se pueda procesar de la mejor forma posible, para posteriormente transformar estos datos en conocimiento para la toma de decisiones, lo cual se traduce en valor real y conocimiento. En este contexto surge el área de investigación conocida como KDD (Knowledge Discovery in Databases), que se define como el proceso no trivial de identificar patrones validos, desconocidos, potencialmente útiles y comprensibles en los datos. Para lograr esto es que se surgen los procesos ETL (Extract, Transform and Load) , relacionados con la obtención de la información de la mejor forma posible, para después ser almacenados en un repositorio físico o virtual. Las herramientas tradicionales para análisis de datos no resultan adecuadas para su recopilación, almacenamiento, gestión y análisis, es en este contexto que surge el concepto de Big Data, que hace referencia a un gran conjunto de datos, cuyas características son su gran volumen,

velocidad, variedad, veracidad y valor de los datos (Battista et al., 2016).

Es en el contexto del análisis de datos, que surgen las técnicas estadísticas avanzadas para modelar situaciones y darle sentido a la información que se quiere procesar, los modelos estadísticos son muy útiles para predecir y clasificar categorías, siendo en la actualidad el Machine learning un conjunto de algoritmos que sirven para modelar diferentes problemáticas explicadas por Dietterich and Bakiri (1994)

Esta investigación, se centrará en el aprendizaje supervisado, donde se le da una variable objetivo al estudio a explicar, cuyo fin es implementar un algoritmo de clasificación, en el cual la variable dependiente es una variable categórica, la que es explicada por un conjunto de variables independientes. Entre los métodos de aprendizaje supervisado mas usados se encuentran: Decision trees , Naive Bayes classifiers, Support Vector Machines, Regresión logística, Neural Network entre otros (Kotsiantis, 2007).

Con la posibilidad de hallar patrones y tendencias en estos grandes volúmenes de datos, se pueden extraer conocimientos que impactan directamente en la toma de decisiones en áreas tan diversas como la astronomía, salud, genética, agronomía, climatología, meteorología, redes sociales, marketing, finanzas, educación, entre otras. Siendo de interés la aplicación en metodologías de gestión de proyectos de minería de datos con un enfoque ágil , en este caso se aplicarán metodologías específicas con el objetivo de comparar características y restricciones de cada una (Dietterich and Bakiri, 1994).

Hoy en día, con la gran variedad de datos, no solo basta con entrenar los modelos, sino que también es necesario conocer técnicas que potencien estos modelos y reduzcan los errores que estos pueden cometer al momento de clasificar un nuevo registro, es por esto que surgen meta-algoritmos llamados Boosting y Bagging, los cuales utilizan un conjunto de algoritmos de clasificación para que trabajen en conjunto, estos pueden estar en forma secuencial o paralelo, los cuales sirven para disminuir el error total de los modelos (Battista et al., 2016).

Existen varios factores que afectan los tiempos de ejecución de un algoritmo de apren-

dizaje automático, entre ellos la capacidad de procesamiento y el volumen de datos con el que se va a trabajar, estos ultimo son el combustible para el aprendizaje de los modelos, lo cuales pueden recibir datos por lotes (Batch), es decir, un conjunto de datos, donde los algoritmos procesan repetidamente los datos de entrenamiento cada vez que se requiere entrenar el modelo, o se pueden trabajar en línea (Streaming), es decir en tiempo real, donde los datos llegan de forma continua, siendo estos de un tamaño indeterminado, es por esto que en estas situaciones donde los datos de entrenamiento son lo suficientemente grandes y costosos de procesar, es que el procesamiento en línea toma ventaja con respecto a los datos en lotes (Beygelzimer et al., 2015).

La principal ventaja de los algoritmos en línea sobre los algoritmos por lotes, es la posibilidad de actualizar incrementalmente los modelos con nuevos ejemplos de entrenamiento, mientras que los algoritmos de aprendizaje por lotes a menudo tienen que olvidar la información previamente aprendida y entrenar un nuevo modelo con los nuevos registros. Esto es claramente mucho desperdicio computacional y es imposible cuando hay más datos de los que se pueden almacenar (Beygelzimer et al., 2015).

1.2. Planteamiento del problema

Con el constante aumento del volumen de datos que nos enfrentamos en la actualidad, surge la necesidad de procesar esta información de la manera más óptima posible para que el costo en recursos computacionales sea el menor posible, con esto surgen los algoritmos en línea, que buscan ajustar modelos estadísticos de forma optimizada. Dado que los modelos necesitan datos para su entrenamiento mediante lotes o en línea, para el caso en lotes es necesario entrenar el modelo con la data de entrenamiento para obtener los parámetros de cada modelo, mientras que en línea llegan datos constantemente los cuales son clasificados e incorporados a los datos totales, aumentando con cada clasificación el tamaño de esta. En esta investigación el objetivo es implementar un algoritmo en línea el cual no tenga

la necesidad de volver a entrenarse cada vez que se le incorporan nuevas observaciones, sino que, dándole un mayor peso al entrenamiento previo, los parámetros del modelo se puedan actualizar a medida que se actualiza la data sin necesidad de un re-entrenamiento total con la data antigua y la nueva, sino que solo procesar los nuevos registros (Freund and Schapire, 1996)

Ensemble learning o aprendizaje conjunto es un paradigma del Machine Learning en el que se entrenan varios modelos débiles o Weak Learners cuya capacidad predictiva es levemente mejor al azar para resolver el mismo problema y así combinarlos para obtener mejores resultados. La hipótesis principal es que cuando los modelos débiles se combinan correctamente, podemos obtener modelos más precisos y / o robustos (Oza, 2000). Entre los métodos de Ensemble Learning se encuentra el Bootstrap Aggregation (Bagging) es un meta-algoritmo, donde dado un conjunto de datos de entrenamiento T de tamaño N , crea modelos en base a m lotes, Cada modelo es entrenado por un algoritmo de aprendizaje por lotes (Batch) donde cada m es obtenida mediante muestreo por reposición (Bootstrap) a la muestra de tamaño N , donde el conjunto de entrenamiento de cada modelo tiene K copias. El método consiste en implementar varios Weak Learners e implementarlos de forma paralela, donde cada algoritmo débil aporta en la toma de decisión final, volviéndose un clasificador fuerte (Dietterich and Bakiri, 1994).

Boosting es otro método para construir un conjunto de Weak Learners de forma secuencial, cuyo fin es reducir la tasa de error en la clasificación. Boosting funciona mediante la construcción iterativa de algoritmos débiles cuyo conjunto de entrenamiento está condicionado al desempeño de los miembros anteriores del conjunto, donde se les da un mayor peso a las clasificaciones erróneas de los algoritmos anteriores, para que el siguiente algoritmo los clasifique de forma correcta. Según Freund and Schapire (1996), un conjunto de algoritmos, toman mejores decisiones que uno por si solo.

Para lograr implementar un algoritmo en línea, se utilizaran los métodos de Ensemble Learning propuestos por Oza (2005), los cuales serán entrenados por lotes y en línea para

comparar sus desempeños, donde el principal objetivo es el algoritmo en línea, ya que dadas todas las problemáticas que generan la gran cantidad de datos y su procesamiento, es el más útil.

Los datos con los que se va a desarrollar este proyecto, serán en primera instancia simulados para probar el funcionamiento de los métodos y posteriormente se probará con un conjunto de datos astronómico o algún conjunto de datos reales que sea interesante e ilustrativo aplicar.

Los algoritmos y análisis serán programados en el software estadístico R (R Core Team, 2019) y Python (Python Software Foundation, 1995) según sea conveniente.

1.3. Objetivos

1.3.1. Objetivo general

- Implementar un algoritmo Ensemble learning siguiendo un enfoque de clasificación en linea.

1.3.2. Objetivos específicos

- Realizar una revisión bibliográfica de los principales métodos de Ensemble learning.
- Revisar los principales algoritmos usados como Weak Learners para problemas de clasificación.
- Proponer los Weak learners que tengan mejor rendimiento .
- Implementar un algoritmo de Ensemble learning por lotes con los mejores Weak Learners.
- Implementar un algoritmo de Ensemble learning siguiendo un enfoque en online.
- Comparar el rendimiento de los algoritmos siguiendo un enfoque en lote y en linea para datos simulados y reales.

Capítulo 2

Marco teórico

2.1. Machine Learning

El Machine Learning o aprendizaje automático es un área de la inteligencia artificial que engloba un conjunto de técnicas que hacen posible el aprendizaje automático a través del entrenamiento con grandes volúmenes de datos (Russo et al., 2016). Para esto se requiere la identificación de patrones complejos dentro de una gran cantidad de datos recopilados mediante ejemplos, experiencia o las instrucciones predefinidas. Tras el concepto de aprendizaje se encuentra un algoritmo que revisa los datos y es capaz de encontrar relaciones o patrones para predecir comportamientos futuros, adaptándose a la incorporación de información adicional y recalibrando los resultados para lograr un mejor desempeño. Esto no quiere decir que el humano no tenga que cumplir ningún papel. Los analistas y programadores son necesarios para participar en tareas como la revisión y confirmación de decisiones críticas, incluso los sistemas mas automatizados, confían en expertos para crear y mantener reglas y monitorizar los resultados. A pesa que el Machine Learning pertenece principalmente al campo de las ciencias de la computación, también requiere de conocimientos estadísticos. Por una parte, la información se centra en resolver como podemos construir maquinas que resuelvan problemas y que problemas son manejables y

tratables y, por otro lado, la estadística se requiere a que se puede deducir de los datos, junto a una serie de suposiciones que son necesarias para un correcto modelado de datos, lo que necesita fuerte base estadística, esto quiere decir, que el aprendizaje automático se encuentra influenciado por ambas disciplinas.

Sobre el Machine Learning sabemos que existen dos grandes enfoques: el aprendizaje supervisado o supervised learning y el aprendizaje no supervisado o unsupervised learning. Esta investigación se enfocara principalmente en aprendizaje supervisado, el cual se basa en entrenar un algoritmo con datos etiquetados para una posterior evaluación de desempeño.

2.2. Aprendizaje supervisado

Los algoritmos supervisados de aprendizaje automático descubren patrones, ideas y relaciones en un conjunto de datos de entrenamiento etiquetado, es decir, un conjunto de datos que ya contiene un valor conocido de la variable objetivo para cada registro. Debido a que proporciona al algoritmo de aprendizaje automático las respuestas correctas para un problema durante el entrenamiento, el algoritmo puede aprender cómo el resto de las variables independientes se relacionan con la variable objetivo, lo que le permite descubrir ideas y hacer predicciones sobre resultados futuros basados en el historial de datos.

2.2.1. Regresión logística

La regresión logística es una herramienta estadística que permite modelar la relación entre una variable de respuesta categórica binaria y un conjunto de variables explicativas continuas y/o categóricas. Entonces tenemos:

$$Y = \begin{cases} 1 & \text{si ocurre el evento} \\ 0 & \text{no ocurre el evento} \end{cases}$$

La v.a $Y \sim \text{Bernoulli}(p)$ por lo que $\mathbb{E}(Y|X) = p$, donde $p \in [0, 1]$, entonces la

función logística para el caso que se tengan k variables independientes queda definida de la siguiente forma:

$$\text{logit}(p) = \ln \left(\frac{\mathbb{P}(Y = 1|X)}{1 - \mathbb{P}(Y = 1|X)} \right) = \ln \left(\frac{p}{1 - p} \right) = \beta_0 + \beta_1 x_1 + \dots + \beta_k x_k$$

De donde se obtiene:

$$\pi(x) = \mathbb{P}(Y = 1|X) = \frac{e^{X^t \beta}}{1 + e^{X^t \beta}} = \frac{1}{1 + e^{-X^t \beta}}$$

la cual es la probabilidad de que ocurra el evento de interés.

Odds y Odds Ratio

- *Odds*:

Los odds se definen como el cociente entre las probabilidades de las dos alternativas, es decir, un Odd es la razón entre la probabilidad de ocurrencia de un evento y la probabilidad de no ocurrencia de dicho evento.

$$\Omega = \frac{\mathbb{P}(Y = 1|X)}{\mathbb{P}(Y = 0|X)} = \frac{\pi(x)}{1 - \pi(x)} = e^{X^t \beta}$$

Aplicando $\ln$ tenemos:

$$\ln \left(\frac{\pi(x)}{1 - \pi(x)} \right) = X^t \beta$$

- *Odds Ratio (OR)*:

Para el caso del modelo de regresión logística, se suele expresar en términos de OR, los cuales están definidos como la razón entre el Odd de exposición al evento y el Odd sin exposición al evento.

$$\psi = \frac{p_1/1 - p_1}{p_2/1 - p_2}$$

De donde p_1 y p_2 representan la probabilidad de ocurrencia de Y dada la presencia de x .

Suponemos tenemos valores de x y x^* , la odds de ocurrencia de la variable respuesta, cuando x toma el valor de x y cuando toma el valor de x^* , se puede escribir, como:

$$\psi = \frac{p_1/1 - p_1}{p_2/1 - p_2} = \frac{e^{\beta_0 + \beta_1 x}}{e^{\beta_0 + \beta_1 x^*}} = e^{\beta_1 x - \beta_1 x^*}$$

El valor del OR es entre $(0, +\infty)$ y su interpretación corresponde a valor que tome, se tiene:

- Cuando el $OR > 1$ indica que el atributo estudiado es más frecuente en el grupo de expuestos que en el no expuestos, se le denomina como un factor de riesgo.
- Un valor $OR = 1$ indica que el atributo se da por igual tanto para los expuestos y no expuestos.
- Un valor $OR < 1$ indica que el atributo estudiado es menos frecuente en el grupo de expuestos que en el grupo de no expuestos, se le denomina como un factor protector.

Estimación de Parámetros Se ocupa el método de máxima verosimilitud para la estimación. La distribución de probabilidades de cada observación es,

$$p_i(Y_i) = \pi_i^{Y_i} (1 - \pi_i)^{1-Y_i}$$

ya que,

- Cada observación de la muestra sigue una distribución Bernoulli.
- La forma general del modelo es: $Y_i = \mathbb{E}(Y_i) + \varepsilon_i$
- Los valores esperados son $\mathbb{E}(Y_i) = \pi_i$

Como las observaciones son independientes, la función verosimilitud es:

$$L(y_1, \dots, y_n, \beta) = \prod_{i=1}^n p_i(Y_i)$$

Las condiciones de primer orden del problema de maximización requieren de las primeras derivadas respecto a β . Dada la no linealidad de los modelos, los estimadores máximo verosímiles se obtienen empleando métodos numéricos como Newton- Raphson (Hosmer Jr et al., 2013).

2.2.2. Árboles de decisión

Esta técnica predictiva de clasificación consiste en una división jerárquica y secuencial del problema en el que cada una de estas divisiones o nodos describen gráficamente las decisiones posibles y por lo tanto los resultados de las distintas combinaciones de decisiones y eventos. A cada evento se le asignan probabilidades y a cada una de las ramas se le determina un resultado. Los árboles representan reglas las que pueden ser comprendidas por los humanos con la ventaja de que el conocimiento lo genera el mismo árbol y no parte de la premisa de un experto en el tema. La mayoría de los algoritmos utilizados para construir un árbol son variaciones de uno genérico llamado “Greedy algorithm” que básicamente va desde la raíz hacia abajo (Top-Down) buscando de manera recursiva los atributos que generan el mejor árbol hasta encontrar el óptimo global con una estructura de árbol lo más simple posible (Chipman et al., 1998).

Las principales características de estos algoritmos es su capacidad de procesar un gran volumen de información de manera eficiente y que pueden manejar de manera óptima el error en los valores o en la clasificación de estos que pudiese existir en los datos de

entrenamiento. En general los distintos modelos se diferencian en el algoritmo que clasifica los distintos atributos del árbol y la eficiencia de estos con el objetivo de obtener un mejor árbol que sea lo más simple posible. Siendo el más conocido el ID3

Algoritmo ID3 (Iterative Dichotomies 3):

Básicamente ID3 construye un árbol de decisión (DT) desde un dataset fijo de “ejemplos”, el DT generado se usa para clasificar futuros ejemplos. Cada ejemplo tiene varios atributos que pertenecen a una clase (como los valores sí o no). Los nodos de “hoja” del árbol (leaf nodes) contienen el nombre de la clase, mientras que los nodos “no-hoja” son los nodos de decisión donde cada uno de ellos (cada rama) corresponde un posible valor del atributo. Cada nodo de decisión es una prueba del atributo con otro árbol que comienza a partir de él. El algoritmo ID3 utiliza el criterio de la “ganancia de información” para decidir que atributo va en cada nodo de decisión. Esta medida estadística mide que tan bien un atributo divide los ejemplos de entrenamiento en cada una de las clases seleccionando aquella con más información (información útil para separar). Para definir “ganancia de información” primero debemos definir el concepto entropía que básicamente corresponde a la cantidad de incertidumbre en un atributo. Dada una colección S (dataset sobre la cual se calcula la entropía que cambia con cada iteración) con c posibles resultados:

$$Entropia(S) = \sum -p_x \log_2 p_x \quad (2.1)$$

Donde x es el dataset de clases en S y p_x es la proporción de S que pertenece a la clase x . Cuando $Entropia(S) = 0$ entonces el dataset S está perfectamente clasificado, es decir, que todos los elementos de S están en la misma clase (si es 1 es que están clasificados en forma aleatoria). De la definición anterior se desprende que la “ganancia de información” (Information Gain “IG”) corresponde a la diferencia en entropía entre antes de haber separado la data S por un atributo versus después de hacerlo, es decir, en cuanto se redujo la incertidumbre en el dataset S después de dividirla en el atributo “A”:

$$IG(S, A) = Entropia(S) - \sum \left(\frac{|S_v|}{|S|} \right) Entropia(x) \quad (2.2)$$

Donde S_v es el set de S para el cual el atributo A tiene el valor v . Los elementos $|S_v|$ y $|S|$ corresponden al número de observaciones en S_v y S respectivamente (Cunha, 2011).

Las principales ventajas de este método: son su interpretabilidad, pues nos da un conjunto de reglas a partir de las cuales se pueden tomar decisiones. Este es un algoritmo que no es demandante en poder de cómputo comparado con procedimientos más sofisticados y, a pesar de ello, que tiende a dar buenos resultados de predicción para muchos tipos de datos.

Sus principales desventajas: son que este es un tipo de clasificación “débil”, pues sus resultados pueden variar mucho dependiendo de la muestra de datos usados para entrenar un modelo. Además es fácil sobre ajustar los modelos, esto es, hacerlos excelentes para clasificar datos que conocemos, pero deficientes para datos conocidos.

2.2.3. Perceptrón Simple

El perceptrón es la forma más simple de una red neuronal usada para la clasificación de un tipo especial de patrones.

Un clasificador de dos dimensiones se puede expresar de la siguiente forma:

$f(x_1, x_2) = b + w_1x_1 + w_2x_2$ Donde w_i son los pesos o Weights asociado a cada variable de entrada como se puede ver en la figura 2.1. Al extenderse a n dimensiones se puede expresar como:

$$f(x) = b + \sum_i^n w_i x_i \quad (2.3)$$

En el caso de la clasificación ,dado el vector de entrada X , la clase predicha está dada por:

$$Y = \begin{cases} 1 & \text{si } f(x) > 0 \\ 0 & \text{si } f(x) < 0 \end{cases}$$

La estructura básica de un perceptrón es la siguiente:

- Inputs: Datos de entrada con los cuales se quiere clasificar.
- Weights: Constantes que multiplican a incógnitas (inputs) de la ecuación.
- Bias: Constante que permite que se tome una decisión.
- Threshold: Punto que representa la división de las clasificaciones.
- Output: Predicción de la clasificación.

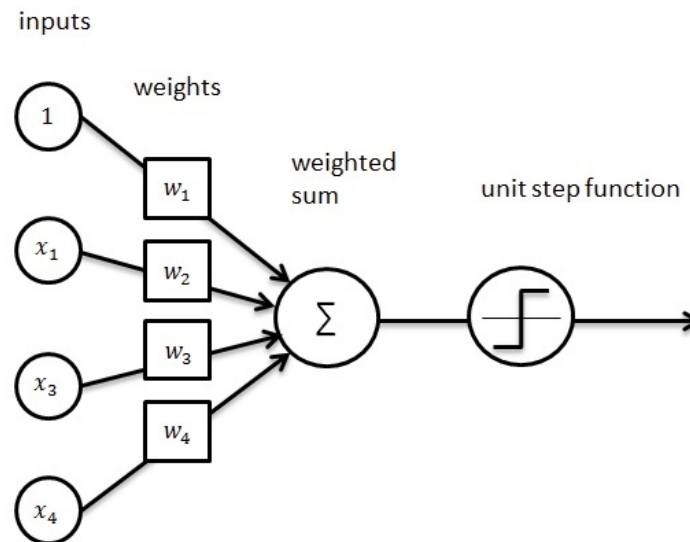


Figura 2.1: Perceptrón de una capa (SONG, 2018) [Figura]. Recuperado de <https://tianyusong.com/>

Podemos interpretar que cada peso w_i representa la influencia relativa de la entrada por la cual se multiplica x_i . A menudo al término b se le llama Sesgo, ya que controla qué tan predispuesta está la neurona a disparar un 1 o un 0 independiente de los pesos. Un sesgo alto hace que la neurona requiera una entrada más alta para generar una salida de 1. Un sesgo bajo lo hace más fácil (Stephen, 1990).

Podemos obtener una verdadera red neuronal a partir de esta fórmula si introducimos dos innovaciones. La primera es la adición de una función de activación, la cual transforma

nuestra discriminador lineal en lo que se llama una neurona, o “unidad” . La segunda innovación consiste en organizar las neuronas de una manera particular: una arquitectura de neuronas conectadas secuencialmente en capas.

Función de activación:

Tanto en las redes neuronales artificiales como biológicas, una neurona no sólo transmite la entrada que recibe. Existe un paso adicional, una función de activación, que es análoga a la tasa de potencial de acción disparando en el cerebro. La función de activación utiliza la misma suma ponderada de la entrada anterior, y la transforma una vez más como salida.

Se han propuesto muchas funciones de activación, pero por ahora describiremos solamente la sigmoide.

Históricamente, la función sigmoide es la función de activación mas antigua y popular. Se define como:

$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad (2.4)$$

donde e denota la constante exponencial, que es aproximadamente igual a 2,71828. Una neurona que utiliza la sigmoide como función de activación se le llama neurona sigmoide. Primero establecemos que la variable z equivale a nuestra suma ponderada de entrada y después la pasamos a través de la función sigmoide.

$$z = b + \sum_i w_i x_i \quad (2.5)$$

$$\sigma(z) = \frac{1}{1 + e^{-z}} \quad (2.6)$$

tiene una forma bastante simple. La podemos ver si trazamos el valor de $\sigma(z)$ como función de la entrada z

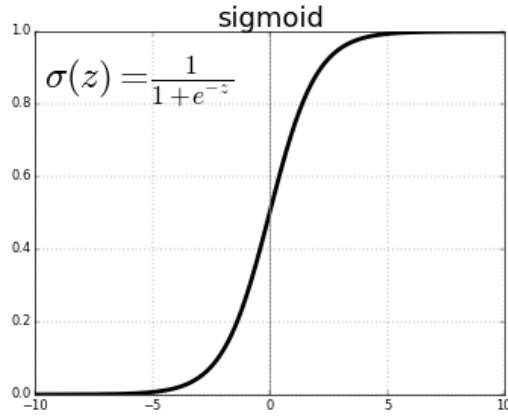


Figura 2.2: Función sigmoide elaboración propia

2.2.4. Naives Bayes

Principio del algoritmo Naives Bayes: Un clasificador Naive Bayes es un modelo probabilístico de aprendizaje automático que se utiliza para la tarea de clasificación o predicción. El principio fundamental del clasificador se basa en el teorema de Bayes.

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)} \quad (2.7)$$

Usando el teorema de Bayes, podemos encontrar la probabilidad de que ocurra A , dado que B ha ocurrido. Aquí, B es la evidencia y A es la hipótesis. La suposición hecha aquí es que los predictores son independientes. Es decir, la presencia de una característica particular no afecta a la otra. Por eso se llama ingenuo (Patil et al., 2013).

El teorema de Bayes se puede reescribir como:

$$P(y|X) = \frac{P(X|y)P(y)}{P(X)} \quad (2.8)$$

La variable y es la variable de objetivo, que en este caso representa si ocurre o no el evento en cuestión dado las otras variables independientes condicionalmente.

X se da como: $X = (x_1, x_2, x_3, \dots, x_n)$ Aquí $x_1, x_2, \dots, x_n$ representan las variables independientes, es decir. Al sustituir por X y expandir usando la regla de la cadena que

obtenemos:

$$P(y|x_1, \dots, x_n) = \frac{P(x_1|y)P(x_2|y)\dots P(x_n|y)P(y)}{P(x_1)P(x_2)\dots P(x_n)} \quad (2.9)$$

Ahora, puede obtener los valores para cada uno mirando el conjunto de datos y sustituirlos en la ecuación. Para todas las entradas en el conjunto de datos, el denominador no cambia, permanece estático. Por lo tanto, el denominador se puede eliminar y se puede introducir una proporcionalidad.

$$P(y|x_1, \dots, x_n) \propto P(y) \prod_{i=1}^n P(x_i|y) \quad (2.10)$$

En nuestro caso, la variable de clase (y) solo tiene dos resultados, sí o no. Podría haber casos en los que la clasificación podría ser multivariada. Por lo tanto, necesitamos encontrar la clase y con la máxima probabilidad.

$$y = \operatorname{argmax}_y P(y) \prod_{i=1}^n P(x_i|y) \quad (2.11)$$

Usando la función anterior, podemos obtener la clase a predecir, dados los variables explicativas.

2.2.5. Ensemble Learning

2.2.5.1. Preliminares:

Weak Learners:

Se define como un Weak Learner o clasificador débil a un modelo de clasificación que al entrenarlo tiene una performance levemente superior a un clasificador aleatorio, es decir un poco mayor 50 %. Algunos ejemplos de clasificadores débiles son los arboles de decisión, perceptrón, regresión logística, entre otros.

Muestreo con reposición: Bootstrap:

La idea básica, es tratar las muestras como si fueran la población, y a partir de ella extraer con reposición un gran número de remuestras de tamaño n . Así, aunque cada remuestra tendrá el mismo número de elementos que la muestra original, mediante el remuestreo con reposición cada una podría incluir algunos de los datos originales más de una vez. Como resultado, cada remuestra será, muy probablemente, algo diferente de la muestra original; con lo cual, un estadística $\hat{\theta}^*$, calculado a partir de unas de esas remuestras tomará un valor diferente del que produce otra remuestra y del $\hat{\theta}$ observado. La afirmación fundamental de bootstrap es que una distribución de frecuencias de esos $\hat{\theta}^*$ calculados a partir de las remuestras es una estimación de la distribución muestral de $\hat{\theta}$.

El bootstrapping es una técnica estadística que consiste en generar muestras de tamaño B (llamadas muestras de bootstrap) a partir de un conjunto de datos inicial de tamaño N mediante el dibujo aleatorio con observaciones de reemplazo B .

Según algunos supuestos, estas muestras tienen propiedades estadísticas bastante buenas : en primera aproximación, pueden verse como extraídas directamente de la distribución de datos subyacente verdadera (y a menudo desconocida) e independientemente de las demás. Por lo tanto, pueden considerarse como muestras representativas e independientes de la verdadera distribución de datos (casi muestras de IID). La hipótesis que debe verificarse para que esta aproximación sea válida es doble. Primero, el tamaño N del conjunto de datos inicial debe ser lo suficientemente grande como para capturar la mayor parte de

2.2.5.2. Modelos de Ensemble Learning

El aprendizaje conjunto es un paradigma de aprendizaje automático en el que varios Weak Learners se entrenan para resolver el mismo problema y se combinan para obtener mejores resultados. La hipótesis principal es que cuando los modelos débiles se combinan correctamente, podemos obtener modelos más precisos y robustos (Oza, 2000).

En el aprendizaje automático, no importa si nos enfrentamos a un problema de clasificación o regresión, la elección del modelo es extremadamente importante para tener alguna posibilidad de obtener buenos resultados. Esta elección puede depender de muchas variables del problema: cantidad de datos, dimensionalidad del espacio, supuestos de distribución, etc... Un sesgo bajo y una varianza baja, aunque a menudo varían en direcciones opuestas, son las dos características fundamentales que se esperan para un modelo. De hecho, para poder modelar un problema, queremos que nuestro modelo tenga suficientes grados de libertad para resolver la complejidad subyacente de los datos con los que estamos trabajando, pero también queremos que no tenga demasiados grados de libertad para evitar Alta variación y ser más robusto. Este es el conocido equilibrio de sesgo-varianza explicado en la figura 2.4.

Un punto importante en la elección de un Weak Learners debe ser coherente con la forma en que agregamos estos modelos . Si se eligen modelos base con bajo sesgo pero alta varianza, debería ser con un método de agregación que tiende a reducir la varianza, mientras que si elegimos modelos base con baja varianza pero alto sesgo, debería ser con un método de agregación que tiende a reducir el sesgo.

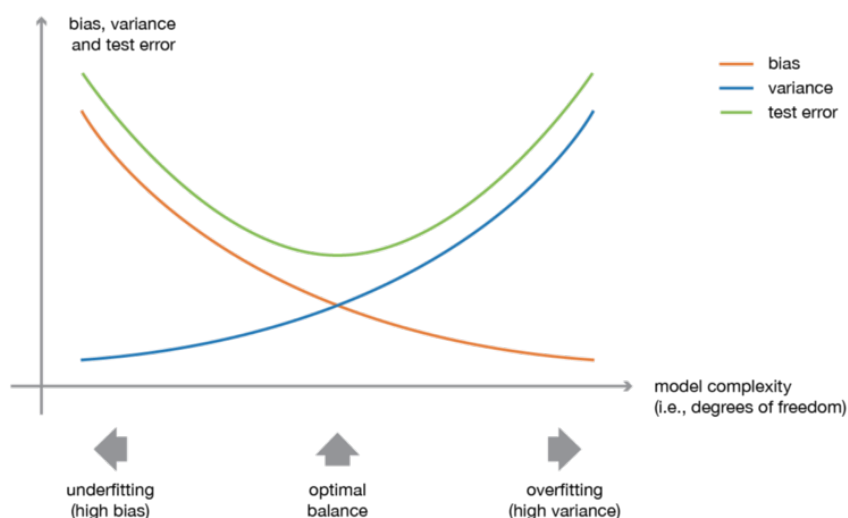


Figura 2.4: Equilibrio Sesgo-Varianza (Rocca, 2019) [Figura]. Recuperado de <https://towardsdatascience.com/>

2.2.5.3. Bagging: Bootstrap Aggregation:

En los algoritmos de Ensemble, el método Bagging forman parte de una clase de algoritmos que construyen varias instancias de un estimador de caja negra en subconjuntos aleatorios del conjunto de entrenamiento original y luego agregan sus predicciones individuales para formar una predicción final. Estos métodos se utilizan como una forma de reducir la varianza de un estimador base o Weak Learners, al introducir la aleatorización en su procedimiento de construcción y luego hacer un conjunto de él. En muchos casos, el método bagging constituyen una forma muy simple de mejorar con respecto a un solo modelo, sin que sea necesario adaptar el algoritmo base subyacente. Como proporciona una forma de reducir el sobre ajuste, bagging funciona mejor que modelos fuertes y complejos (por ejemplo, árboles de decisión completamente desarrollados).

Los métodos Bagging vienen en diferentes tipos, pero en su mayoría difieren entre sí por la forma en que extraen subconjuntos aleatorios del conjunto de entrenamiento:

- Cuando los subconjuntos aleatorios del conjunto de datos se extraen como subconjuntos aleatorios de las muestras, este algoritmo se conoce como Pasting (Breiman, 1999).
- Cuando se extraen muestras con reemplazo, el método se conoce como Bagging (Breiman, 1996).
- Cuando los subconjuntos aleatorios del conjunto de datos se dibujan como subconjuntos aleatorios de las características, entonces el método se conoce como Random Subspaces (Barandiaran, 1998) .
- Finalmente, cuando los estimadores básicos se basan en subconjuntos de muestras y características, el método se conoce como Random Patches (Louppe and Geurts, 2012).

donde en particular el Bagging funciona de la siguiente forma:

Dado un conjunto de datos de entrenamiento T de tamaño N el bagging estándar crea M Modelos bases o Weak Learners . Cada modelo se entrena con un algoritmo L_b mediante lotes con una muestra de tamaño N creado al extraer muestras aleatorias con reemplazo del conjuntos de entrenamiento original. h_m son las predicciones del modelo entrenado con T_m muestras. El pseudo código se ve explicado en la figura 2.5

```

Bagging( $T, L_b, M$ )
  For each  $m \in \{1, 2, \dots, M\}$ ,
     $T_m = \text{Sample\_With\_Replacement}(T, |T|)$ 
     $h_m = L_b(T_m)$ 
  Return  $\{h_1, h_2, \dots, h_M\}$ 

```

Figura 2.5: Batch Bagging (Oza, 2005)

2.2.5.4. Boosting

Otra técnica utilizada en los métodos de Ensemble Learning y capaz de construir conjuntos de clasificadores fuertes es Boosting (Freund et al., 1999), es un algoritmo adaptativo en el que cada clasificador se construye en base a los resultados obtenidos en los clasificadores previos mediante la asignación de pesos a cada uno de los ejemplos de entrenamiento, por ejemplo, patrones que han sido incorrectamente clasificados por los clasificadores anteriores tendrán mas importancia a la hora de construir un nuevo clasificador. De esta forma, se puede decir que los clasificadores se centran en aquellos ejemplos que son mas difíciles de etiquetar correctamente por el conjunto de variables Boosting es uno de los mejores algoritmos para construir conjuntos de clasificadores, esto debido a que combina dos efectos: reduce el sesgo del clasificador base debido a que los fuerza a no cometer los mismos errores y reduce la varianza al combinar mediante voto mayoritario distintas hipótesis. Otra característica que se debe considerar es que presenta problemas de generalización en presencia de datos con outliers ya que los clasificadores se centran en los ejemplos difíciles sin tener en cuenta si son datos correctos o no. También debe destacar que este método es serializable, ya que cada clasificador se construye en base a los resultados obtenidos por el clasificador previo siendo estando cada modelo en serie. en la figura 2.6 se puede apreciar la diferencia entre un modelo, bagging y boosting.

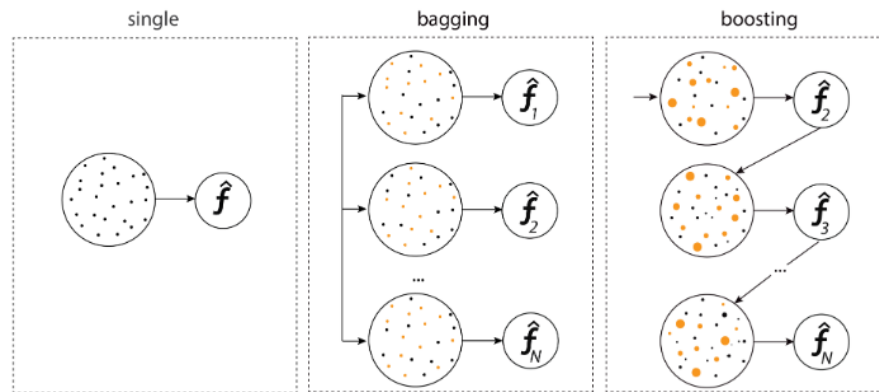


Figura 2.6: Comparación entre un modelo base, Bagging y Boosting [Figura].

2.3. Online Ensemble Learning

2.3.1. Tipos de Aprendizaje

2.3.1.1. Aprendizaje Batch o Lote:

En el aprendizaje tipo batch o en lote, también llamado aprendizaje offline, el modelo se entrena con todos los datos disponibles, lo que se hace antes de realizar ningún tipo de predicción. Si los datos de entrenamiento cambian, por ejemplo, si se reciben nuevos datos, el modelo deberá ser reentrenado desde cero (Ferrer et al., 2005).

2.3.1.2. Aprendizaje Online o Streaming:

En el aprendizaje online, el modelo se entrena incrementalmente con cada una de las nuevas muestras que se reciban, o en grupos pequeños de muestras llamados mini-batches (o mini-lotes).

Estos sistemas son más adecuados en entornos en los que los datos a partir de los que se entrena el algoritmo cambian con cierta rapidez. También son especialmente útiles en escenarios en los que no es posible cargar todos los datos en la memoria del ordenador, estos escenarios son denominados “out-of-core learning”.

En este tipo de aprendizaje resulta crítica la velocidad con la que el modelo deberá adaptarse a los nuevos datos, parámetro al que se denomina tasa de aprendizaje o learning rate. Una mayor tasa de aprendizaje llevará al modelo a olvidar más rápidamente los datos previamente considerados y, al contrario, una menor tasa de aprendizaje supondrá una mayor inercia y llevará al modelo a olvidar los datos antiguos más despacio (Kuncheva, 2008).

En realidad, en ambos enfoques el entrenamiento se realiza “offline”, es decir, se entrena el modelo y solo después es posible realizar predicciones a partir de él, por lo que el nombre de “aprendizaje online” puede resultar confuso.

2.3.2. Bagging Online

Oza Bagging es un método de aprendizaje conjunto presentado por primera vez por 'Online Bagging and Boosting' de Oza y Russel. Son una mejora del conocido método de ensacado Bagging para la configuración de lotes, que en esta versión puede manejar de manera efectiva los flujos de datos (Oza, 2000).

Para un algoritmo de Bagging tradicional, adaptado para la configuración del lote, tendríamos M clasificadores entrenando en M diferentes conjuntos de datos, creados al extraer N muestras del conjunto de entrenamiento de tamaño N con reemplazo.

En el contexto en línea, dado que no hay un conjunto de datos de entrenamiento, sino un flujo de muestras, la extracción de muestras con reemplazo no se puede ejecutar trivialmente. La estrategia adoptada por el algoritmo bagging en línea es simular esta tarea entrenando cada K muestra que llega, que se extrae de la distribución binomial. Como podemos considerar que el flujo de datos es infinito, y sabiendo que con muestras infinitas la distribución binomial tiende a una distribución de *Poisson*(1), Oza y Russel descubrieron que era una buena “extracción con reemplazo”.

Siguiendo paso a paso, ya que cada conjunto de entrenamiento del modelo base contiene k copias de cada uno de los ejemplos de entrenamiento originales donde:

$$P(K = k) = \binom{N}{k} \left(\frac{1}{N}\right)^k \left(\frac{N}{k}\right)^{N-k} \quad (2.12)$$

La que es la distribución binomial, cuando $N \rightarrow \infty$ la distribución de K tiende a una *Poisson*(1), tal que $P(K = k) = \frac{e^{-1}}{k!}$, a medida que cada ejemplo de entrenamiento $d = (x, y)$ se presente a nuestro algoritmo, para cada modelo base, elija el ejemplo $K \sim \text{Poisson}(1)$ veces y actualice el modelo base en consecuencia utilizando el algoritmo en línea base. Los nuevos ejemplos se clasifican de la misma manera online y por lotes, mediante votación no ponderada de los M modelos.

```

OnlineBagging( $\mathbf{h}, L_o, d$ )
  For each base model  $h_m \in \mathbf{h}, m \in \{1, 2, \dots, M\}$ ,
    Set  $k$  according to Poisson(1).
    Do  $k$  times
       $h_m = L_o(h_m, d)$ .

```

Figura 2.7: Online Bagging (Oza, 2005)

2.3.3. Complejidad Algorítmica

2.3.3.1. Notación Big O

La complejidad del algoritmo es independiente de la velocidad del computador en el que se ejecuta y se define utilizando la notación Big-O. La idea es conocer cuántas operaciones se necesitan a medida que aumentan los datos de entrada (Chivers and Sleightholme, 2015)

- $O(1)$: constante. La operación no depende del tamaño de los datos. Es el caso ideal, pero a la vez probablemente el menos frecuente. No se ve en la gráfica de más abajo porque la logarítmica le pasa justo por encima y la tapa.
- $O(n)$: lineal. El tiempo de ejecución es directamente proporcional al tamaño de los datos. Crece en una línea recta.
- $O(\log n)$: logarítmica. por regla general se asocia con algoritmos que "trocean.^{el} problema para abordarlo, como por ejemplo una búsqueda binaria.
- $O(n \log n)$: en este caso se trata de funciones similares a las anteriores, pero que rompen el problema en varios trozos por cada elemento, volviendo a recomponer información tras la ejecución de cada "trozo". Por ejemplo, el algoritmo de búsqueda Quicksort.
- $O(n^2)$: cuadrática. Es típico de algoritmos que necesitan realizar una iteración por todos los elementos en cada uno de los elementos a procesar. Por ejemplo el algoritmo

de ordenación de burbuja. Si tuviese que hacer la iteración más de una vez serían de complejidad $O(n^3)$, $O(n^4)$, etc... pero se trata de casos muy raros y poco optimizados.

- $O(2^n)$: exponencial. Se trata de funciones que duplican su complejidad con cada elemento añadido al procesamiento. Son algoritmos muy raros pues en condiciones normales no debería ser necesario hacer algo así. Un ejemplo sería, por ejemplo, el cálculo recursivo de la serie de Fibonacci, que es muy poco eficiente (se calcula llamándose a sí misma la función con los dos números anteriores: $F(n)=F(n-1)+F(n-2)$)).
- $O(n!)$: explosión combinatoria. Un algoritmo que siga esta complejidad es un algoritmo totalmente fallido. Una explosión combinatoria se dispara de tal manera que cuando el conjunto crece un poco, lo normal es que se considere computacionalmente inviable. Solo se suele dar en algoritmos que tratan de resolver algo por la mera fuerza bruta.

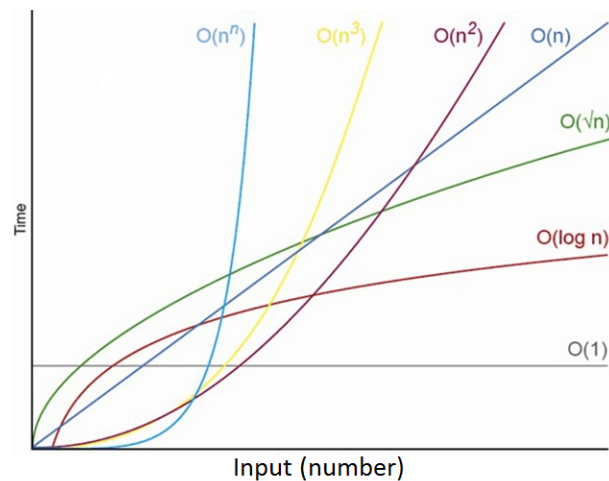


Figura 2.8: Comparación de tipos de complejidad [Figura]. Recuperado de <https://notas.uristolar.com/complejidad-algoritmica>

Las complejidades de los Weak Learners son las siguientes:

- **Regresión Logística:** La complejidad teórica de la regresión logística es $O(np)$, donde n es la cantidad de datos de entrada y p en numero de variables (Tsangaratos and Ilia, 2016)
- **Árbol de decisión:** La complejidad de un arbol de clasificacion binario con un nivel de profudidad es $O(np\log(n))$ (Tsang et al., 2005).
- **Naive Bayes:**La complejidad de Naive Bayes es de $O(np)$ (Tsang et al., 2005).
- **Perceptrón:**Supongamos que hay muestras n muestras de entrenamiento, m variables, y k capas ocultas, cada una con h neuronas , por simplicidad, y o neuronas de salida. La complejidad temporal de la propagación hacia atrás es $O(nmh^ki)$, dónde i es el número de iteraciones (Pedregosa et al., 2011).

2.4. Métricas de clasificación

2.4.1. Matriz de confusión

Para estudiar las diferentes métricas de clasificación es necesario definir la Matriz de Confusión, que es una forma tabular de resumir la información sobre los éxitos y fracasos en la predicción de los modelos. Cabe mencionar, que categoriza las predicciones de acuerdo a si coinciden con el valor real de la categoría (Visa et al., 2011). En este caso, se analizará a un problema de clasificación binaria, donde los valores tomados por la variable objetivo son 0 y 1, donde 1 hace referencia a que sí posee el atributo y 0 lo contrario, es decir que no lo posee. Se considera Matriz de Confusión a aquella matriz de 2×2 cuyas filas corresponden a los valores observados y cuyas columnas corresponden a los valores predichos por un modelo de clasificación. De forma ilustrativa, se tiene la siguiente matriz:

		Predicciones	
		0	1
Observado	0	TN	FP
	1	FN	TN

Cuadro 2.1: Matriz de confusión

- TN: Correctamente clasificado como negativo.
- TP: Correctamente clasificado como positivo.
- FN: Incorrectamente clasificado como negativo.
- FP: Incorrectamente clasificado como positivo.

A partir de esta matriz, en las secciones posteriores se entenderá de mejor manera cada una de las medidas de rendimiento que serán aplicadas a los diversos modelos implementados, dado que esta matriz es la base de la mayor parte de ellos.

2.4.1.1. Acurracy

La exactitud o Accuracy se define como el cociente entre el numero de casos clasificados correctamente entre el total de casos. Representa el porcentaje de acierto del modelo.

mide la frecuencia con la que el clasificador hace la predicción correcta. Es la relación entre el número de predicciones correctas y el número total de predicciones (el número de puntos de datos de prueba)

$$Acurracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (2.13)$$

Esta es la métrica mas sencilla y la mas utilizada, pero a su vez resulta algo inexacta, ya que no tiene en cuenta la distribución del error entre las clases. Ello conlleva que en una muestra donde el 90 % de los casos son 2 el resto de los casos son ", cualquier modelo que obtenga un Accuracy alrededor de 90 %, tendera a ser considerado como un modelo aceptable, cuando en realidad puede que este clasificando todos los ejemplos como buenos. Por lo que, en los casos que el valor del Accuracy sea menor que el mayor porcentaje de la clase mas repetida, se dirá que el modelo no aporta ningún tipo de conocimiento. Cabe destacar, que son estas razones por lo que se recomienda evaluar un modelo mediante mas métricas de discriminación.

2.4.1.2. Precisión

La precisión nos dice qué proporción de eventos clasificamos como una cierta clase, en realidad era esa clase. Es una proporción de verdaderos positivos a todos los positivos. Respondiendo a ¿Qué proporción de identificaciones positivas fue correcta?

$$Precisión = \frac{TP}{TP + FP} \quad (2.14)$$

2.4.1.3. Sensibilidad o Recall

nos dice qué proporción de eventos que en realidad eran de cierta clase fueron clasificados por nosotros como esa clase. Es una proporción de verdaderos positivos a todos los positivos. Respondiendo a ¿Qué proporción de positivos reales se identificó correctamente?.

$$Recall = \frac{TP}{TP + TN} \quad (2.15)$$

2.4.1.4. Especificidad

La especificidad es correctamente etiquetada por el programa para todos los que en realidad son saludables. Specifity responde a la siguiente pregunta: De todas las personas que están sanas, ¿cuántas de ellas predijimos correctamente?

$$Especificidad = \frac{TN}{TN + FP} \quad (2.16)$$

2.4.2. Curva ROC (receiver operating characteristic curve)

Constituye un método estadístico para determinar la exactitud diagnóstica de un determinado test que utilizan escalas continuas, siendo utilizadas con tres propósitos específicos: determinar el punto de corte en el que se alcanza la sensibilidad y especificidad más alta, evaluar la capacidad discriminativa del test diagnóstico, es decir, su capacidad de diferenciar positivos versus negativos, y comparar la capacidad discriminativa de dos o más tests diagnósticos que expresan sus resultados como escalas continuas (Patil et al., 2013).

2.4.3. KS (Kolmogórov-Smirnov)

Kolmogorov Smirnov es una medida de separación que busca medir cuan distintas son dos funciones de distribución. Para esto, calcula la máxima diferencia en valor absoluto de las distribuciones empíricas utilizando las frecuencias relativas acumuladas de cada muestra en estudio. Si se considera la distribución de positivos y negativos para un problema

dado, y que además los datos están agrupados en K categorías, entonces se calculan las frecuencias relativas acumuladas F^i y G^i con $i = 1; \dots; k$, para luego calcular las diferencias en valor absoluto (Nieto, 2010).

$$KS = \max \left| \widehat{F}_{bueno} - \widehat{G}_{malos} \right| \quad (2.17)$$

El valor que arroja la formula descrita, corresponde al estadístico KS, el cual oscila entre 0 y 1. El limite inferior corresponde a un clasificador que no es capaz de discriminar entre positivos y negativos, mientras que el limite superior corresponde al caso en el que el clasificador discrimina de manera perfecta entre las dos clases existentes.

2.4.4. Índice GINI

El índice GINI es una medida de discriminación propuesta por Corrado Gini, la cual mide la desigualdad en una distribución. En sus inicios fue utilizada principalmente para medir la desigualdad de ingresos entre países, sin embargo, se ha aplicado en otros contextos de discriminación, como lo es la estadística. Este índice oscila entre 0 y 1, y se calcula comparando la cantidad acumulada de casos buenos y malos por tramos de variable o score de un modelo. Gráficamente, es el área comprendida entre las dos líneas en la curva (XYZ), expresada como el porcentaje máximo posible (XYW). Estos ejes en el gráfico representan el porcentaje acumulado de buenos (eje Y) y el porcentaje acumulado de malos (eje X). Por lo tanto, la relación sigue la siguiente expresión:

$$GINI = 2AUC - 1 \quad (2.18)$$

Ahora bien, como valores de referencia para medir el poder de discriminación de un clasificador mediante el coeficiente GINI, este debe ser mayor a un 60 %, el cual indicara un poder discriminación razonable.

2.4.5. Norma F1 Score

Existe un trueque entre Precision y Recall, en la medida que ambos puntajes se pueden promediar para obtener una medida ponderada de ambos fenómenos. Esta medida se conoce como F1, y representa la media armónica entre Precisión y Recall. Se prefiere la media armónica dado que penaliza de mayor manera los valores bajos. F1 alcanza un máximo de 1 cuando Precisión y Recall son 1, y 0 cuando Precisión o Recall son 0, aún cuando una de las dos medidas sea 1. Se utiliza ésta dado que pondera de mayor manera a los valores bajos. Esto genera el F1 sea alto sólo cuando precisión y recall lo sean.

$$F1 - Score = \frac{2 * Precisión * Recall}{Precisión + Recall} \quad (2.19)$$

Capítulo 3

Implementación software

Para la aplicación de los modelos propuestos, se usara el package RMOA (Massive Online Analysis) de R (Wijffels, 2014) y la library Scikit-Multiflow (Multi-output Streaming Framework) (Montiel et al., 2018) de Python inspirada en RMOA. En estos paquetes se puede implementar los modelos descritos anteriormente, a modo ilustrativo, se hará una pequeña demostración del funcionamiento de las funciones mostraran el modelo Bagging con enfoque online de forma general en Python y R.

3.1. Python

3.1.1. Scikit-learn

Scikit-learn es una biblioteca de aprendizaje automático de código abierto que admite el aprendizaje supervisado y no supervisado. También proporciona varias herramientas para el ajuste de modelos, el pre-procesamiento de datos, la selección y evaluación de modelos, entre muchas otras utilidades (Pedregosa et al., 2011). Para separar los datos de entrenamiento y de prueba, se utiliza la función *train_test_split*, definiendo la matriz de atributos X y el vector objetivo Y

```
X_train, X_test, y_train, y_test, = train_test_split(X, y, test_size = 0.3,  
random_state = 63)
```

donde los argumentos de salida son:

- **X_train:** Corresponde a la matriz X de atributos, siendo el 70 % del total según se especifique. Siendo esta la matriz utilizada para entrenar los modelos.
- **X_test:** Corresponde al 30 % restante de la matriz de atributos X , siendo esta matriz utilizada para hacer predicciones y testear el rendimiento del modelo.
- **y_train:** Corresponde al 70 % del vector objetivo Y , el cual tiene las clases con que se van a entrenar el modelo.
- **y_test:** Corresponde al 30 % del vector objetivo Y , siendo utilizada con la matriz X de test para evaluar el rendimiento real del modelo de clasificación.

Posteriormente, para definir el modelo que se pondra a prueba, se usan las siguientes funciones ingresando como parámetros la matriz X_{train} y el vector y_{train} . Con la función *model.fit* se pueden hacer estimaciones con el modelo definido usando los datos de entrenamiento.

con la función *model.predict* se pueden obtener predicciones usando los datos de test definidos anteriormente. La función *model.score* obtiene la precisión del modelo al clasificar las clases, se puede obtener cualquier métrica dependiendo si es un problema de clasificación o regresión. A continuación se muestra el uso de los 4 modelos usados como Weak learners.

- **Regresión Logística:**

```
model = linear_model.LogisticRegression()
```

```
model.fit(X_train,y_train)
predictions = model.predict(X_test)
model.score(X_test,y_test)
```

- **Árbol de decisión:**

```
model_arbol = tree.DecisionTreeClassifier()
model_arbol.fit(X_train,y_train)
predictions = model_arbol.predict(X_test)
model_arbol.score(X_test,y_test)
```

- **Naive Bayes:**

```
model_nbayes = GaussianNB()
model_nbayes.fit(X_train, y_train)
predictions = model_nbayes.predict(X_test)
model_nbayes.score(X_test,y_test)
```

- **Perceptron simple:**

```
percep = Perceptron(tol=1e-3, random_state=0)
percep.fit(X_train,y_train)
predictions = percep.predict(X_test)
percep.score(X_test,y_test)
```

También se pueden usar las funciones para modelos de Ensemble Learning como Bagging y Boosting, los cuales funcionan de forma similar especificando los datos de entrenamiento y de test. En estos modelos hay que ingresar como parámetro el modelo base y el numero de modelos que se utilizaran.

- **Bagging:**

```
model_bag = BaggingClassifier(model_nbayes,  
n_estimators=1000,bootstrap = True, random_state = 1)  
model_bag.fit(X_train,y_train)  
predictions = model_bag.predict(X_test)  
model_bag.score(X_test,y_test)
```

Donde el parámetro *bootstrap* es para indicar si se hará un muestreo con reposicion o no.

- **Boosting:**

```
model_boos = AdaBoostClassifier(base_estimator=model_nbayes, n_estimators=800,  
learning_rate=1.5, random_state=1)  
model_boos.fit(X_train,y_train)  
predictions = model_boos.predict(X_test)  
model_boos.score(X_test,y_test)
```

Donde el parámetro *learning_rate* es para indicar la tasa de aprendizaje que se utilizara.

3.1.2. Scikit-Multiflow (Multi-output Streaming Framework)

Scikit-multiflow está inspirado en MOA , el marco de código abierto más popular para el aprendizaje automático con flujos de datos (Montiel et al., 2018). Scikit-multiflow también está inspirado en scikit-learn , el marco más popular para el aprendizaje automático en Python. Siguiendo la filosofía de SciKits , scikit-multiflow es un marco de aprendizaje automático de código abierto para múltiples categorías y datos de transmisión. Al estar inspirado en scikit-learn, se pueden usar los modelos y sus respectivos parámetros.

A continuación se mostraran las principales funciones para el uso de datos con enfoque en línea y para el entrenamiento de modelos de Ensamble Learning.

3.1.2.1. FileStream

FileStream (Stream):

Crea una secuencia desde un origen de archivo. Por el momento, solo se admiten archivos CSV, pero el objetivo es admitir diferentes formatos, siempre que haya una función que lee, interpreta y devuelve correctamente el DataFrame de pandas o numpy.ndarray con los datos.

Parámetros:

- **Ruta de archivo:** Ruta al archivo de datos.
- **target_idx:** int, opcional (*predeterminado* = -1) El índice de columna a partir del cual comienzan los objetivos.
- **n_targets:** int, opcional (*predeterminado* = 1) El número de objetivos.
- **cat_features:** list, opcional (*predeterminado* = *Ninguno*) Una lista de índices correspondientes a la ubicación de características categóricas.

Notas: El objeto del flujo de datos proporciona, a pedido, una cantidad de muestras, de tal manera que no se puede acceder a muestras antiguas en otro momento. Esto se hace para simular correctamente el contexto del flujo.

3.1.2.2. Evaluation

evaluation.EvaluatePrequential (`n_wait=200`, `max_samples=100000`, `batch_size=1`, `pretrain_size=200`, `max_time=inf`, `metrics=None`, `output_file=None`, `show_plot=False`, `restart_stream=True`, `data_points_for_classification=False`):

El método de evaluación secuencial o el método intercalado de prueba y entrenamiento es una alternativa a la evaluación tradicional de holdout, heredada de los problemas de configuración de lotes.

La evaluación previa está diseñada específicamente para configuraciones de flujo, en el sentido de que cada muestra tiene dos propósitos, y que las muestras se analizan secuencialmente, en orden de llegada, y se vuelven inmediatamente inaccesibles.

Este método consiste en usar cada muestra para probar el modelo, lo que significa hacer predicciones, y luego se usa la misma muestra para entrenar el modelo (ajuste parcial). De esta manera, el modelo siempre se prueba en muestras que aún no ha visto.

Parametros:

- **n_wait (int (Default: 200))** El número de muestras para procesar entre cada prueba. También define cuándo actualizar el gráfico si `show_plot = True`. Tenga en cuenta que establecer `n_wait` demasiado pequeño puede ralentizar significativamente el proceso de evaluación.
- **max_samples (int (Default: 100000))** – El número máximo de muestras para procesar durante la evaluación.

- **batch_size** (**int** (**Default: 1**)) – El número de muestras a pasar a la vez a los modelos.
- **pretrain_size** (**int** (**Default: 200**)) – El número de muestras a usar para entrenar el modelo antes de comenzar la evaluación. Se utiliza para imponer un inicio “cálido”.
- **max_time** (**float** (**Default: float(“inf”)**)) – La duración máxima de la simulación (en segundos).
- **metrics** (**list**, **optional** (**Default: ['accuracy', 'kappa']**)) –
La lista de métricas a seguir durante la evaluación. También define las métricas que se mostrarán en gráficos y / o se registrarán en el archivo de salida. Las opciones válidas son: accuracy, kappa, true_vs_predicted, precision, recall, f1.
- **output_file** (**string**, **optional** (**Default: None**)) – nombre del archivo para guardar el resumen de la evaluación.
- **show_plot** (**bool** (**Default: False**)) – si es Verdadero, un gráfico mostrará el progreso de la evaluación. Advertencia: el trazado puede ralentizar el proceso de evaluación.
- **restart_stream** (**bool**, **optional** (**default: True**)) – si es True, la secuencia se reinicia una vez que se completa la evaluación.
- **data_points_for_classification** (**bool**(**Default: False**)) – si es True, la visualización utilizada es una nube de puntos de datos (solo funciona para la clasificación) y se ignoran las métricas de rendimiento predeterminadas. Si se requieren métricas específicas, se deben establecer explícitamente con el atributo metrics

Notas: Este evaluador puede procesar a un solo alumno para rastrear su desempeño; o múltiples alumnos a la vez, para comparar diferentes modelos en la misma secuencia.

La métrica `true_vs_predicted` está destinada a ser solo informativa. Corresponde a evaluaciones en un momento específico que podrían no representar el rendimiento real del alumno en todas las instancias.

3.1.2.3. Ensemble methods

Oza Bagging:

`meta.OzaBagging(base_estimator= “ ”, n_estimators=, random_state=“ ”)`

Parámetros:

- **base_estimator:** Modelo base a ser usado como Weak Learners.
- **n_estimators:** Numero de muestras a entrenar por cada modelo base.
- **random_state:** Es el generador de números aleatorios, para poder replicar los resultados.

3.1.3. Simulación

Para simular una situación real en donde se puedan utilizar estas metodologías, se proba un dataset con números aleatorios para simular un conjunto de datos de entrada a un modelo. El conjunto de datos tiene como vector objetivo una variable binaria y las clases se encuentran equilibradas para simplificar la aplicación con el software estadístico.

Cuadro 3.1: Muestra de datos simulados

X1	X2	X3	X4	Y
640	47	158	11	1
425	43	156	9	1
227	57	169	17	0
...	...	...	...	...
542	86	147	4	0

3.1.3.1. Weak Learners

Para poder aplicar las principales funciones de Scikit-learn para métodos de Machine Learning, se simulo un dataset aleatorio el cual servirá de base para la implementación de modelos. El dataset como se muestra en la figura 3.1 esta compuesto de 4 variables continuas o numéricas y el vector objetivo una variable categórica binaria.

Usando las funciones de Scikit-learn se pueden implementar los 4 modelos de clasificación candidatos a posibles Weak Learners para los modelos de Ensemble Learning. Los

datos se dividirán en un 70 % de datos de entrenamiento y 30 % de validación. La metrica de referencia sera el Accuracy, pero se puede utilizar Recall o F1-Score.

Cuadro 3.2: Accuracy Weak Learners simulados

	Accuracy	Precision	Recall	F1-score
Regresión logística	50,02 %	49,39 %	49,44 %	38,37 %
Árbol de decisión	50,08 %	49,77 %	49,75 %	49,74 %
Naive Bayes	50,35 %	51,25 %	50,35 %	44,08 %
Perceptrón Simple	50,48 %	25,49 %	50,48 %	33,87 %

3.1.3.2. Complejidad Algorítmica

A continuación, se simulara la complejidad de los principales Weak Learners, con datos simulados en conjuntos de 500, 1.000, 2.000, 5.000, 10.000, 20.000 y 40.000 registros, los que son utilizados para entrenar los modelos con 1, 5, 10, 15 y 20 variables, logrando una aproximación de las complejidad usando la notación Big O, comparando con su complejidad teórica.

Se puede observar en los gráficos con datos simulados, donde el eje X representa el numero de datos de entrenamiento y el eje Y representa tiempo que se demora en procesar los datos, donde las curvas representan el numero de variables con que se entreno el modelo.

Los modelos se ajustan a su complejidad-tiempo, con sus respectivas notaciones Big (O), siendo el mas eficiente y eficaz Naive bayes.

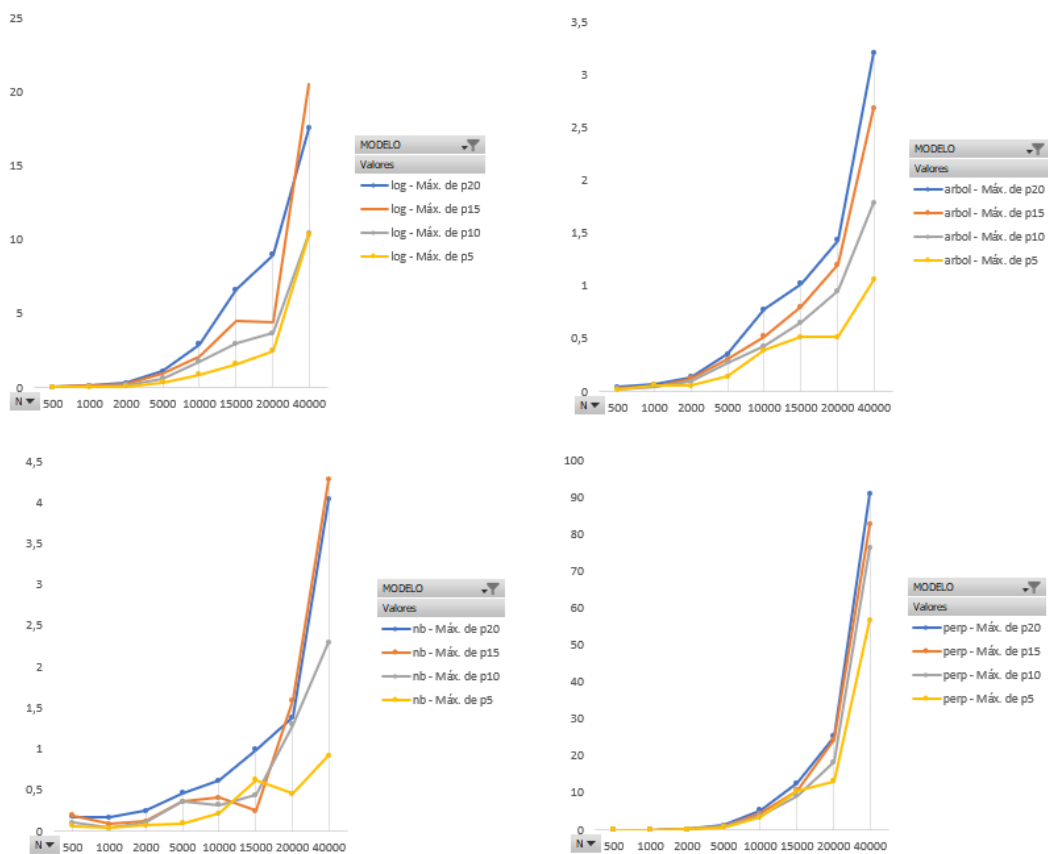


Figura 3.1: Comparación aproximación complejidad tiempo entre los Weak Learners

A continuación, se puede comparar la complejidad empírica de cada Weak Learners para 20 variables.

Cuadro 3.3: Tiempos simulados

N	arbol	log	nb	perp
500	0,042996	0,070009	0,182999	0,017
1000	0,072996	0,162012	0,174994	0,087005
2000	0,141001	0,325111	0,251998	0,244004
5000	0,36	1,143513	0,464	1,244998
10000	0,783008	2,973002	0,622	5,440674
15000	1,023	6,631003	0,999001	12,740514
20000	1,435001	8,988006	1,391004	25,285916
40000	3,207001	17,535	4,039001	91,036515

3.1.3.3. Ensemble Learning

- Bagging

```

model_bag = BaggingClassifier(model_nbayes,
n_estimators=1000,
bootstrap = True, random_state = 1)
model_bag.fit(X_train,y_train)
predictions = model_bag.predict(X_test)
model_bag.score(X_test,y_test)

```

En el modelo Bagging con Naive bayes como modelo base, se usaran 1000 Weak Learners

- Boosting

```

model_boos = AdaBoostClassifier(base_estimator=model_nbayes, n_estimators=800,
learning_rate=1.5, random_state=1)
model_boos.fit(X_train,y_train)
predictions = model_boos.predict(X_test)
model_boos.score(X_test,y_test)

```

En el modelo Boosting, se usara Naive Bayes como Weak Learners, se usaran 800 modelos bases y una tasa de aprendizaje de 1.5, los cuales vienen por defecto y se pueden cambiar para mejorar el rendimiento del modelo.

Cuadro 3.4: Ensembles simulados

	Acurracy	Precisión	Recall	F1-score
Naive Bayes	50,05 %	49,44 %	49,45 %	38,39 %
Bagging	50,09 %	49,72 %	49,77 %	49,73 %
Boosting	50,24 %	51,21 %	50,40 %	44,12 %

3.1.3.4. Online Ensemble Learning

Para entrenar los modelos de Ensemble Learning con enfoque en linea, se usaran las funciones del paquete Scikit-Multiflow. Los datos se importan en formato CSV y para ambos modelos se usara como Weak Learners el modelo Naives Bayes.

- Bagging Online:

Lo primero es definir el modelo base o Weak Learners, y el numero de estimadores, que en este caso serán 10.

```
nb1=NaiveBayes()
clf2 = OzaBagging(base_estimator=nb1, n_estimators=10)
```

Posteriormente con la función *EvaluatePrequential* especificar cuantos ejemplos se usaran para pre-entrenamiento y la métrica utilizada. En este caso el modelo se pre-entrenara con 1000 registros y los mini batch llegaran de forma secuencial en grupos de 100, la métrica con la que se trabajara sera el Acurracy.

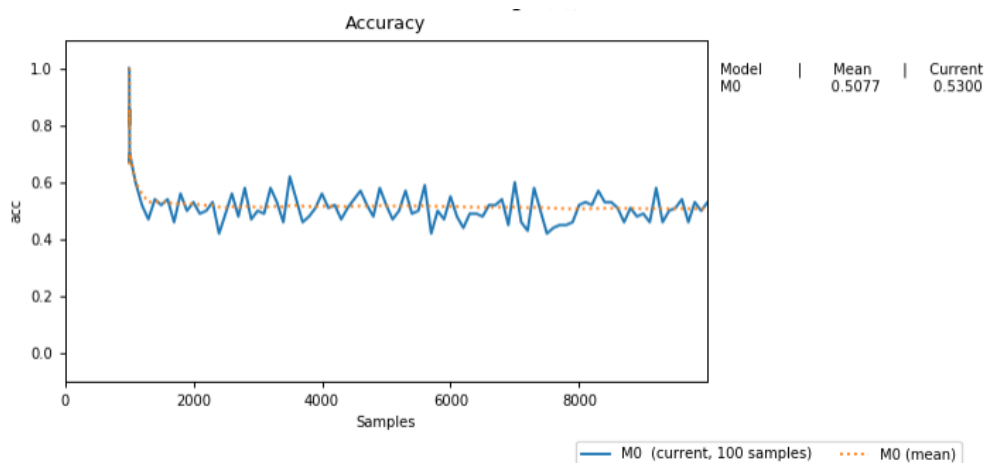


Figura 3.2: Bagging Online

■ Boosting Online:

```
nb1=NaiveBayes()
clf2 = OnlineBoosting(base_estimator=nb1, n_estimators=10)
```

En el caso del modelo Online Boosting, se usaran los mismo parametros que el Online Bagging a modo ilustrativo.

```
evaluator = EvaluatePrequential(show_plot=True,
n_wait = 100, max_samples=10000, metrics=['accuracy'])
evaluator.evaluate(stream=stream, model=clf2)
```

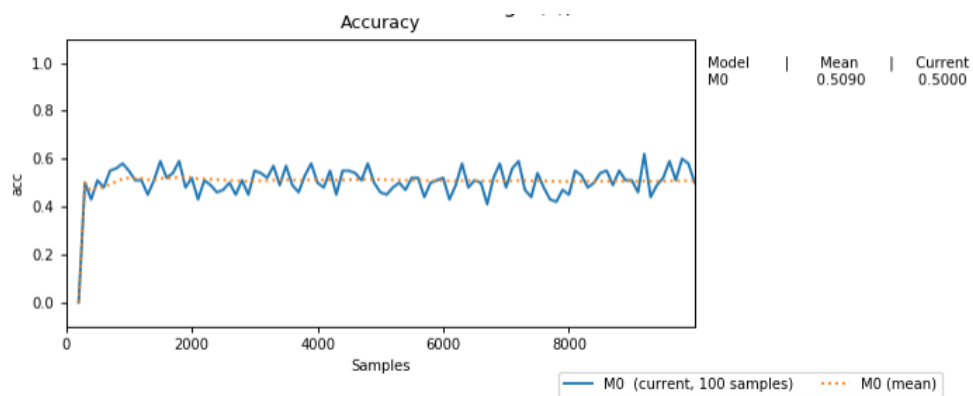


Figura 3.3: Boosting Online

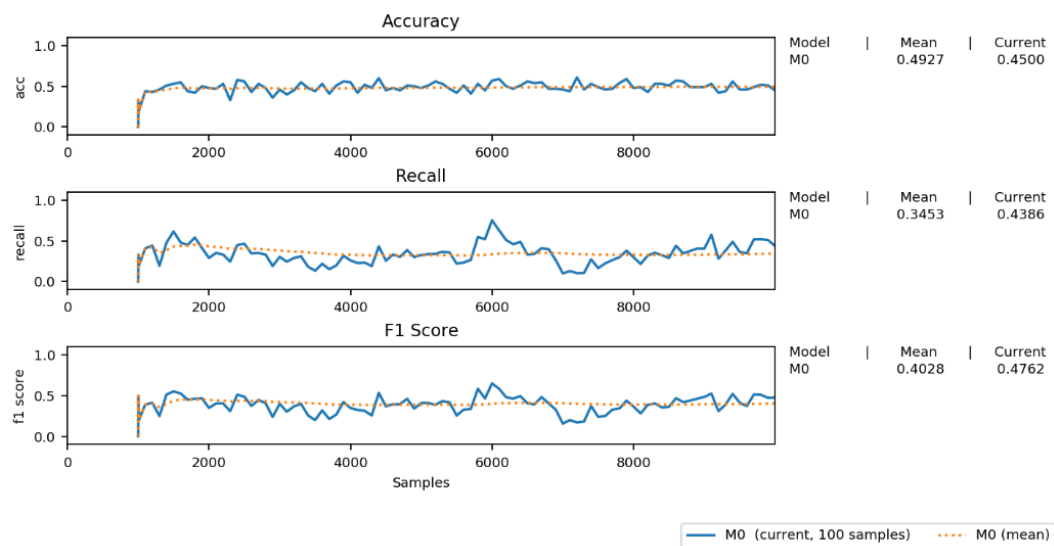


Figura 3.4: Comparación métricas simulados

3.2. R Project

3.2.1. Caret

El Paquete Caret (short for Classification And Regression Training) es un conjunto de funciones que tratan de simplificar el proceso para la creación de modelos predictivos. El paquete contiene herramientas para ,división de datos pre-procesamiento, selección de características entre otras funcionalidades.

Es posible separar los datos de entremiento y de test de forma similar que en Python, especificando el porcentaje para los datos de entramiento y los de test.

```
smp_size <- floor(0.7 * nrow(pulsar_stars))
set.seed(123)
train_ind <- sample(seq_len(nrow(pulsar_stars)), size = smp_size)
dataTrain <- pulsar_stars[train_ind,]
dataTest <- pulsar_stars[-train_ind,]
```

Los modelos tienen la siguiente estructura, especificando en el parámetro *Method* el modelo a utilizar y en *TrControl* si el modelo ajustara sus parámetros con Crossvalidation. También se especifica la forma de obtener la matriz de confusión para ver las predicciones correctas e incorrectas de los modelos.

■ Regresión Logística:

```
glm_model <- train(target_class~.,
                   dataTrain, method = "glm",
                   trControl = trainControl(
                     method = "cv", number = 10,
                     verboseIter = FALSE
                   )
)
```

```

)
#Predicciones#
pred<- predict(glm_model,dataTest)
summary(pred)
#matriz de confusion
confusionMatrix(dataTest$target_class,pred)

```

■ Árbol de decisión:

```

tree_model <- train(target_class~.,
                    dataTrain, method = "rpart",
                    trControl = trainControl(
                        method = "cv", number = 10,
                        verboseIter = FALSE
                    )
)
pred_class2 <- predict(tree_model, dataTest)
confusionMatrix(pred_class2, dataTest$FUGA)

```

■ Naive Bayes:

```

Naivebayes <- train(target_class ~ .,                                dataTrain,
                    method = "naive_bayes",
                    verbose = FALSE)

sms_test_pred=predict(Naivebayes,dataTest)

```

```
confusionMatrix(sms_test_pred,dataTest$target_class)
```

- **Perceptron simple:**

```
perceptron <- train(target_class ~ .,                                dataTrain,
                    method = "Perceptron",
                    verbose = FALSE)
```

```
perceptron_simple_test_pred=predict(perceptron_simple,dataTest)
confusionMatrix(perceptron_simple_test_pred,dataTest$target_class)
```

3.2.2. RMOA (Massive Online Analysis)

MOA significa análisis Masive online Analysis, es un packages de código abierto que permite construir y ejecutar experimentos de aprendizaje automático o minería de datos en flujos de datos en evolución o incrementalmente. Contiene algoritmos de aprendizaje automático para tareas de clasificación, regresión, agrupación, detección de valores atípicos y recomendaciones .

En ocasiones al trabajar con R, nos encontramos con una gran cantidad de datos o encuentran problemas de RAM al construir modelos en grandes conjuntos de datos, RMOA y, en general, los flujos de datos tienen algunas características interesantes. A saber:

- Utiliza una cantidad limitada de memoria. Esto significa que no hay problemas de RAM al construir modelos .
- Procesa un ejemplo a la vez y lo ejecutará solo una vez

- Funciona de forma incremental , para que un modelo esté directamente listo para ser utilizado con fines de predicción

El paquete RMOA y está disponible en github(<https://github.com/jwijffels/RMOA>).

Oza Bagging:

```
mymodel <- OzaBag(baseLearner = "", ensembleSize = "")
mymodel <- trainMOA(model = mymodel, formula = "", data = "")
```

Parámetros:

- **baseLearner:** modelo base a ser usado como Weak Learners.
- **ensembleSize:** Numero de muestras a entrenar por cada modelo base.
- **trainMOA:**Entrena el modelo base mediante datos en linea, especificando el dataset que se utilizara

3.2.3. Simulación

Al igual que en Python, se comparara las métricas de clasificación los 4 Weak learners candidatos a modelos bases

Cuadro 3.5: Weak Learners simulados en R

	Acurracy	Precision	Recall	F1-score
Regresión logística	50,03 %	49,42 %	49,41 %	38,39 %
Árbol de decisión	50,08 %	49,76 %	49,76 %	49,74 %
Naive Bayes	50,35 %	51,26 %	50,33 %	44,12 %
Perceptrón Simple	50,49 %	25,48 %	50,49 %	33,89 %

En el cuadro 3.6 se observa un resumen con las principales métricas de clasificación obtenidas desde R

Cuadro 3.6: Ensembles simulados en R

	Acurracy	Precisión	Recall	F1-score
Naive Bayes	50,06 %	49,45 %	49,46 %	38,40 %
Bagging	50,08 %	49,73 %	49,76 %	49,72 %
Boosting	50,24 %	51,20 %	50,41 %	44,11 %

Capítulo 4

Aplicación problema de clasificación binario: Estrella Pulsar

4.1. Información del conjunto de datos

HTRU2 es un conjunto de datos de observaciones astronómicas, las cuales contienen posibles candidatos de estrellas púlsar recolectados durante la Encuesta del Universo de Alta Resolución de Tiempo (Sur) (Lyon et al., 2015).

Los pulsares son un tipo raro de estrella de neutrones que producen emisiones de radio detectables aquí en la Tierra. Son de considerable interés científico como sondas del espacio-tiempo, el medio interestelar y los estados de la materia.

A medida que los púlsares giran, su haz de emisión recorre el cielo, y cuando cruza nuestra línea de visión, produce un patrón detectable de emisión de radio de banda ancha. A medida que los púlsares giran rápidamente, este patrón se repite periódicamente. Por lo tanto, la búsqueda de púlsar implica buscar señales de radio periódicas con radiotelescopios grandes.

Cada púlsar produce un patrón de emisión ligeramente diferente, que varía ligeramente con cada rotación. Por lo tanto, una detección de señal potencial conocida como 'candidato'

se promedia en muchas rotaciones del púlsar, según lo determinado por la duración de una observación. En ausencia de información adicional, cada candidato podría describir un púlsar real. Sin embargo, en la práctica, casi todas las detecciones son causadas por interferencia de radiofrecuencia (RFI) y ruido, lo que hace que sea difícil encontrar señales legítimas.

Las herramientas de aprendizaje automático ahora se utilizan para etiquetar automáticamente a los candidatos de pulsar para facilitar un análisis rápido. Los sistemas de clasificación en particular se están adoptando ampliamente, que tratan los conjuntos de datos candidatos como problemas de clasificación binaria. Aquí, los ejemplos legítimos de púlsar son una clase positiva minoritaria, y los ejemplos espurios son la clase negativa mayoritaria. Actualmente, las etiquetas multiclase no están disponibles, dados los costos asociados con la anotación de datos.

El conjunto de datos compartido aquí contiene 16259 ejemplos espurios causados por RFI / ruido, y 1639 ejemplos de púlsar reales. Todos estos ejemplos han sido verificados por anotadores humanos.

Los datos se presentan en dos formatos: CSV y ARFF (utilizados por la herramienta de minería de datos WEKA). Los candidatos se almacenan en ambos archivos en filas separadas. Cada fila enumera las variables primero, y la etiqueta de clase es la entrada final. Las etiquetas de clase utilizadas son 0 (negativo) y 1 (positivo).

Información del atributo:

Cada candidato se describe mediante 8 variables continuas y una variable de clase única. Los primeros cuatro son estadísticas simples obtenidas del perfil de pulso integrado (perfil plegado). Este es un conjunto de variables continuas que describen una versión resuelta en longitud de la señal que ha sido promediada en tiempo y frecuencia. Las cuatro variables restantes se obtienen de manera similar de la curva DM-SNR. Estos se resumen a continuación:

- Media del perfil integrado
- Desviación estándar del perfil integrado.
- Exceso de curtosis del perfil integrado.
- Inclinación del perfil integrado.
- Media de la curva DM-SNR.
- Desviación estándar de la curva DM-SNR.
- Exceso de curtosis de la curva DM-SNR.
- Inclinación de la curva DM-SNR.
- Class

Resumen de clases HTRU	
ejemplos positivos	1639
ejemplos negativos	16259
datos totales	17898

Cuadro 4.1: Resumen HTRU

4.2. Aplicación

El principal objetivo es implementar un algoritmo de clasificación que logre categorizar los posibles candidatos a estrellas pulsar y cuales no son candidatos.

4.2.1. Análisis Descriptivo

Cuadro 4.2: Resumen descriptivo HTRU

	mean	std	min	P25	P50	P75	max
Media del perfil integrado	111,1	25,7	5,8	100,9	115,1	127,1	192,6
Desviación estándar del perfil integrado	46,5	6,8	24,8	42,4	46,9	51,0	98,8
Exceso de curtosis del perfil integrado	0,5	1,1	-1,9	0,3	0,2	0,5	8,1
Inclinación del perfil integrado	1,8	6,2	-1,8	-0,2	0,2	0,9	68,1
Media de la curva DM-SNR	12,6	29,5	0,2	1,9	2,8	5,5	223,4
Desviación estándar de la curva DM-SNR	26,3	19,5	7,4	14,4	18,5	28,4	110,6
Exceso de curtosis de la curva DM-SNR	8,3	4,5	-3,1	5,8	8,4	10,7	34,5
Inclinación de la curva DM-SNR	104,9	106,5	-2,0	35,0	83,1	139,3	1191
target_class	0,09	0,29	0	0	0	0	1

4.2.2. Modelos Batch

A continuación se prueba el rendimiento de los modelos Naive bayes, Bagging y Boosting usando como modelos base Naive Bayes, ya que este es el que tiene los tiempos mas acotados de ejecución en comparación con los otros Weak learners.

Se puede observar que los modelos tienen un rendimiento similar, siendo el Boosting el modelo con mejor Accuracy. Cabe mencionar que a pesar de que Boosting tiene un mejor métrica al momento de clasificar, es el modelo que tiene los tiempos mas prolongados de entrenamiento.

Cuadro 4.3: Comparación métricas Weak Learners con enfoque Batch Pulsar

	Accuracy	Precision	Recall	F1-score	Tiempo (Seg)
Regresión logística	96,46 %	96,51 %	96,32 %	96,38 %	0,32
Árbol de decisión	95,03 %	95,69 %	95,23 %	95,11 %	0,43
Naive Bayes	96,63 %	96,60 %	96,62 %	44,08 %	0,006
Perceptrón Simple	94,79 %	95,01 %	94,88 %	94,82 %	0,02

Cuadro 4.4: Comparación Bagging y Boosting con enfoque Batch Pulsar

	Accuracy	Precision	Recall	F1-score	Tiempo (Seg)
Naive Bayes	96,62 %	96,59 %	96,62 %	96,61 %	0,006
Bagging	96,63 %	96,60 %	96,62 %	96,60 %	5,062
Boosting	96,75 %	95,63 %	96,52 %	95,33 %	8,319

4.2.3. Modelos Online

A continuación se entrenaran los modelos con enfoque Online, los cuales se comparan a nivel de Accuracy y tiempo de ejecución. Para esto se utilizaran el simulador de Stream con un pre-entrenamiento de 500 registros y mini-lotes de 100 registros hasta completar el

Cuadro 4.5: Comparación Batch y Online Pulsar

	Acurracy	Recall	Tiempo Online (Seg)	Tiempo Batch (seg)
Bagging	94,23 %	95,37 %	1158	789
Boosting	94,44 %	95,79 %	2132	1406

total de datos, estos mini-lotes de extraen de forma aleatoria sin reposición y se entrena el modelo con cada lote que ingresa al modelo, ajustándose a medida que el flujo de datos de entrada va llegando.

Capítulo 5

Aplicación problema clasificación multiclase: Proyecto Alerce

5.1. Información del conjunto de datos

Como muchos otros campos de investigación, los datos astronómicos han experimentado un crecimiento exponencial durante los últimos años. Por ejemplo, el Gran Telescopio para Rastreo o Sondeos Sinópticos o LSST, el cual estará completamente operativo para el 2022 y escaneará el cielo regularmente produciendo aproximadamente 30 TB de datos por noche, abriendo enormes posibilidades para una nueva ciencia, especialmente poder considerar el dominio del tiempo. Los telescopios de inspección producen millones de alertas por noche donde los objetos cambian o aparecen, lo que plantea la necesidad de sistemas masivos de procesamiento de alertas que lo cuales deberían ejecutarse en tiempo real. Una alerta proveniente de un momento astronómico donde una observación cambia significativamente entre dos tiempos de observación, o épocas. Las alertas generalmente se producen restando una imagen de plantilla de una imagen científica. La imagen científica corresponde a la observación en la época actual, y la imagen de la plantilla o referencia corresponde a una aproximación para el cielo estático. La tardía generalmente se elige co-

mo un promedio adicional de un conjunto de imágenes en diferentes épocas. Si el brillo de una fuente en una coordenada particular varía en el tiempo, luego aparecerá en la imagen de diferencia, mientras los objetos no-variados aparecerán casi por igual en las imágenes científicas y de plantilla, por lo tanto, no aparecerán en la imagen de diferencia.

El proyecto Automatic Learning for the Rapid Classification of Events, Alerce, es un sistema integrado de telescopios de última generación, que apunta a la clasificación rápida de eventos a partir de datos temporales y así obtener candidatos relevantes para el seguimiento. Actualmente ALerCE está procesando la secuencia de alerta pública Zwicky Transient Facility (ZTF), que incluye fotometría g y r. En esta aplicación solo proporcionamos una breve descripción del proceso de construcción de la curva de luz. La canalización ALerCE procesa los archivos de alerta Avro ZTF (<https://zwickytransientfacility.github.io/ztf-avro-alert/>). Estos archivos contienen metadatos e información contextual para un solo evento, que son definidos como flux-transient y flux-variable u objeto en movimiento.

Para la construcción de la curva de luz, ALerCE utiliza la fotometría de la imagen de diferencia y la imagen de referencia (detecciones), posibles no detecciones asociadas al objetivo en los 30 días anteriores del evento (magnitud 5σ límite en la imagen de diferencia basada en la fotometría PSF-fit, llamada como difmaglim por ZTF), el puntaje verdadero-falso de calidad informado por ZTF (rb; que varía de 0 a 1, donde más cercano a 1 es más confiable), y el Clasificación morfológica del objeto más cercano obtenido de PanSTARRS1. Además, la ALERTA utiliza la fotometría W1, W2 y W3 obtenida de la base de datos pública WISE.

5.1.1. Zwicky Transient Facility (ZTF):

El clasificador ALerCE utiliza un total de 136 variables, de las cuales 126 se calculan utilizando los datos públicos de bandas g y r de ZTF (Bellm et al., 2019).

Las variables que se obtienen utilizando las magnitudes observadas se denominan variables

de detección (108 en total), y las variables calculadas utilizando los límites de magnitud de no detección 5σ (diffmaglims) se denominan variables de no detección (20 en total). También incluimos como variables las coordenadas galácticas de cada objetivo (gal b y gal l), los colores W1-W2, W2-W3 WISE y los colores g-W2, g-W3, r-W2 y r-W3, donde g y r son calculados como la magnitud media de las curvas de luz gband y rband para una fuente dada. Adicionalmente, Utilizamos la información incluida en los datos de Avro les: el parámetro sgscore, que corresponde a un puntaje morfológico *Star/Galaxy* de la fuente más cercana de PanSTARRS1 reportado en el ZTF Avro, con $0 \leq sgscore \leq 1$ donde más cercano a 1 implica una mayor probabilidad de ser una estrella; y la mediana del parámetro rb (real-falso) para cada banda. En este trabajo solo consideramos curvas de luz con más de 5 épocas. Si una fuente dada tiene más que cinco épocas en una sola banda, se incluye en el análisis y las variables asociadas a la banda que falta se considera como -999 o NaN (según el modelo de clasificador). Esta regla se aplica a todas las funciones utilizadas por el clasificador, siempre que una función no esté disponible para un objetivo determinado, suponemos un valor igual a -999 o NaN.

5.1.1.1. Variables de Detección

La mayoría de las características utilizadas por nuestro clasificador se calculan utilizando las magnitudes observadas en las bandas g y r. Hay 52 variables diferentes que se calculan para cada banda, y dos características que son calculadas usando una combinación de ambas bandas, produciendo un total de 106 características de detección. Nuestro clasificador utiliza 22 funciones que provienen del Análisis de funciones para series temporales usando el paquete FATS de Python (Nun et al., 2015). Las que corresponden a amplitud, prueba de Anderson-Darling (AndersonDarling), longitud de autocorrelación (Auto_len), Beyond1Std, Con, Índice de variabilidad η (Eta e), Gskew, Tendencia lineal (Tendencia_lineal), Máx Pendiente (MaxSlope), varianza media (Meanvariance), desviación absoluta media (MedianAbsDev), Median en rango de construcción (MedianBRP), tendencia

de pendiente del par (PairSlopeTrend), amplitud porcentual (percent_Amplitude), Q31, Rango de una suma acumulativa en la curva de luz plegada en fase ψ_{CS} (Psi-CS_v2), ψ_ν (Psi.eta_v2), Rango de suma acumulativa R_{cs} (Rcs), Inclinación (Skew), Kurtosis pequeña (SmallKurtosis), Desviación estándar (Std) y Stetson K (StetsonK). Las variables que terminan con “_ 1” se calculan para gband y las funciones que terminan en “_ 2” se calculan para la banda r, lo que da un total de 44 funciones. Además, el clasificador utiliza variables que no están incluidas o están definidas de manera diferente en el Paquete de FAT (Nun et al., 2015).

- **g-r color:** Se obtiene usando los picos de las curvas de luz en cada banda ($g-r_{max}$), y usando los valores medios de las curvas de luz en cada banda ($g-r_{mean}$).
- **positive_fraction_1(_2):** fracción de detecciones positivas en las diferencia de imágenes en la banda g (rband).
- **delta_mag_fid_1 (fid_2):** diferencia entre la magnitud máxima y mínima observada en el gband (rband).
- **MHAOV Period:** Período obtenido usando el paquete P4J python. Para obtener una estimación mas rápida del período en el que utilizamos el modelo MHAOV disponible para P4J. "Periodo_MHAOV_1z "Period_MHAOV_2z corresponde al período calculado para gband y rband respectivamente.
- **Period fit:** Para tener una estimación cruda de la confianza del período candidato desarrollamos un heurística basada en la entropía de los picos del periodograma. Este valor se calcula tomando los 100 valores más grandes del periodograma, luego normalizándolos para sumar 1 y luego el se calcula la entropía de ese vector. Como la entropía puede tomar valores entre cero y $\log(100)$
- **Period Power Rate:** Relación entre el poder del periodograma obtenido para el mejor período candidato (P) versus el poder del periodograma obtenido por 2 ve-

ces este período (tasa = Potencia (P) = Potencia (2 P)). PeriodPowerRate 1 (2) corresponde a la tasa de potencia del período calculado para la gband (rband)

- **Harmonics:** Las series armónicas se usan comúnmente para modelar curvas de luz periódicas. En este trabajo se ajusta una serie armónica hasta el séptimo armónico, de acuerdo con la expresión

$$y_{t_j} = \sum_{i=1}^7 (A_i \cos(\frac{2\pi t_j}{P}) + B_i \sin(\frac{2\pi t_j}{P})) + C \quad (5.1)$$

donde t_j corresponde al tiempo de observación de la detección j -ésima, P es el mejor candidato para el período calculado por la característica 'Periodo MHAOV' y y_{t_j} es la magnitud estimada por los armónicos modelo. A_i , B_i y C para $i = 1; \dots; 7$ se obtienen minimizando el error cuadrático medio entre Las magnitudes observadas y el modelo. Se puede notar que el modelo es lineal con respecto a sus parámetros, para que puedan ser calculados usando regresión lineal ponderada. Las inversas de la los cuadrados de cada error observacional se usan como pesos, para dar menos importancia en el regresión a las observaciones con mayor ruido. Luego, la ecuación 2.2 se reescribe como

$$y_{t_j} = \sum_{i=1}^7 M_i \cos(\frac{2\pi t_j}{P} - \phi) + C \quad (5.2)$$

donde $M_i = \sqrt{A_i^2 + B_i^2}$ y $\phi_i = \arctan(\frac{B_i}{A_i})$ El modelo se desplaza en el tiempo para tener fase cero en el primer armónico, que se realiza siguiendo la expresión $\phi'_i = \phi_i - i\phi_1$. Finalmente, el parámetros M_i para $i = 1, \dots, 7$, ϕ' para $i = 2, \dots, 7$ como el error cuadrático medio se usan como variable que reciben los nombres Harmonics_mag_1, ..., Harmonics_mag_7, Harmonics_phase_2, ..., Harmonics_phase_7 y Harmonics_mse, respectivamente.

Estas variables se calculan independientemente para la banda g y r, que se denota en el nombre de la variable con 1 o 2 extra como en otras variables. .

- **P_var:** Probabilidad de que la fuente sea intrínsecamente variable en una banda dada. Considera los χ^2 de la curva de luz, y calcula la probabilidad $P_{var} = P(\chi^2)$ de que χ^2 sea menor o igual al observado el valor podría ocurrir por casualidad para una fuente intrínsecamente no variable. corresponden al Pvar_1 calculado para gband y rband respectivamente.
- **Excess Variance (σ_{rms}):** Medida de la amplitud de variabilidad intrínseca en una banda dada. $\sigma_{rms}^2 = \frac{\sigma_{LC}^2 - \sigma_m^2}{m^2}$ donde σ_{LC}^2 es la desviación estándar de la curva de luz, m es el error fotométrico promedio, y $\bar{m}$ es la magnitud promedio. ExcessVar_1 y ExcessVar_2 corresponden a los σ_{rms} : calculados para gband y rband respectivamente.
- **Irregular autoregressive (IAR) model:** Este modelo fue introducido por Eyheramendy et al. (2018) el cual es una representación en tiempo discreto del modelo continuo autorregresivo de orden 1 (CAR (1)), que El desafío de clasificación ALeRCE 7 tiene propiedades estadísticas deseables, como la estacionariedad estricta y la ergodicidad sin una distribución suposición. El modelo IAR está definido por

$$y_{t_j} = \phi^{t_j - t_{j-1}} y_{t_{j-1}} + \sigma \sqrt{1 - \phi^{2(t_j - t_{j-1})}} \varepsilon_{t_j} \quad (5.3)$$

donde ε_{t_j} es una secuencia de ruido blanco con media cero y varianza unitaria, es la desviación estándar de y_{t_j} y t_j son los tiempos de observación para $j = 1, ..n$. Utilizamos una versión modificada del Modelo IAR, que considera la varianza conocida de los errores de medición $\delta_{t_j}^2$ en la probabilidad del modelo. Por lo tanto, al asumir una distribución gaussiana, menos la probabilidad logarítmica del proceso es dado por,

$$L(\theta) = \frac{n}{2} \log(2\pi) + \frac{1}{2} \sum_{j=1}^n \log(\nu_{t_j}) + \frac{1}{2} \sum_{j=1}^n \frac{e_{t_j}^2}{\nu_{t_j}} \quad (5.4)$$

Donde $e_{t1} = y_{t1}$, $\nu_{t1} = \sigma_{t1}^2$, $e_{tj} = y_{tj} - \hat{y}_{tj}$ y $\nu_{tj} = \sigma^2(1 - \phi^{2(t_j - t_{j-1})) + \delta_{tj}^2$ para $j = 2, \dots, n$. Particularmente, usamos la estimación de máxima verosimilitud del parámetro ϕ como una variable (obtenida directamente de las curvas de luz gband (rband), y la llamamos IAR_phi_1 (_2) ”.

- **ClassALeRCE:** Para este problema de clasificación, se tienen registros de alertas astronómicas transitorias y variables, siendo las variables clasificadas en periódicas y estocásticas. Estas alertas se clasifican en 14 sub-clases de fenómenos astronómicos. Se consideran las clases: QSO, AGN, Blazar, CV / Nova, YSO, SNIa, SNIbc, EB, DSCT, RRL, Ceph, LPV, Periódico-Otro como se muestra en la figura 5.1.

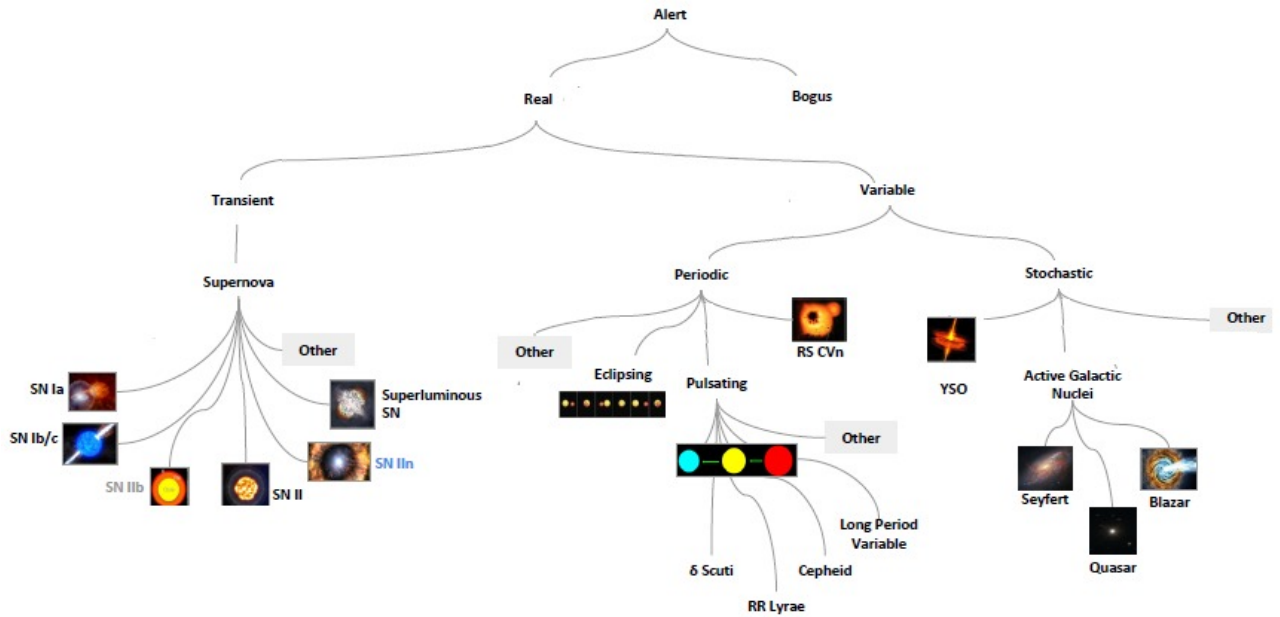


Figura 5.1: Taxonomía definida en colaboración ALeRCE

Cuadro 5.1: Resumen ZTF

Resumen clases ZTF			
EB	11383	Blazar	335
LPV	10297	SNIa	272
RRL	10159	CV/Nova	270
QSO	3814	DSCT	225
AGN	805	Ceph	193
YSO	514	SNII	78
Periodic-Other	340	SNlbc	17

Dado el gran numero de variables obtenidas de ZTF, se tiene que seguir una estrategia de limpieza de datos y de selección de variables para reducir el tiempo de procesamiento y lograr seleccionar el modelo mas parsimonioso posible dada la naturaleza del problema que se espera modelar.

	Mean	Std	Min	P25	P50	P75	Max
IAR_phi1	618597	422902	0	63385	880151	988848	1000000
IAR_phi2	611559	424865	0	37315	867065	989537	1000000
g-r_mean	0,988	1063773	-1719590	0,217	0,516	1850155	6011124
g-r_max	0,922	1088168	-1659080	0,097	0,471	1800926	5840322
Rcs2	0,292	0,097	0,064	0,214	0,291	0,371	0,496
min_mag_2	15569934	1809419	10516517	14189855	15402381	16899120	20532553
Autocor_length2	2645324	3685064	1000000	1000000	1000000	3000000	57000000
paps_PN_flag_2	0,441	0,497	0,000	0,000	0,000	1000000	1000000
Rcs_1	0,281	0,102	0,049	0,197	0,280	0,364	0,499
Autocor_length1	2741829	3524354	1000000	1000000	1000000	3000000	38000000
Pvar_1	927900	227117	0	1000000	1000000	1000000	1000000
Meanvariance1	0,016	0,014	0,000	0,007	0,012	0,019	0,149
positive_fraction1	0,484	0,351	0,000	0,163	0,464	0,800	1000000
Pvar2	924248	235631	0	1000000	1000000	1000000	1000000
min_mag_1	16492430	1652582	11488517	15326724	16584840	17732403	21507551
SF_ML_amplitud	0,800	1935322	-0,500	0,161	0,298	1	15000000
AndersonDarling	0,921	0,183	0,115	0,982	1,000	1000000	1000000
SF_ML_gamma1	0,258	0,513	-0,500	-0,014	0,107	0	3000000
mean_mag_2	15946847	1775481	12026514	14577569	15830259	17275053	20951188

5.1.2. Modelos Batch:

A continuación se realiza una comparación de los Weak Learners utilizados para medir el rendimiento de cada uno.

Para los modelos de Ensemble Learning se realizó un ajuste para obtener el número de modelos bases que maximiza la métrica de clasificación, se utiliza el gráfico de la figura

Cuadro 5.2: Comparación métricas Weak Learners con enfoque Batch ALerCE

	Accuracy	Precision	Recall	F1-score	Tiempo (Seg)
Regresión logística	95,64 %	94,41 %	94,67 %	94,52 %	0,3338
Árbol de decisión	96,04 %	96,01 %	96,02 %	96,02 %	0,3570
Naive Bayes	91,37 %	94,63 %	91,37 %	92,47 %	0,0080
Perceptrón Simple	88,49 %	89,07 %	88,48 %	88,19 %	0,0200

5.2 que se muestra a continuación para encontrar la convergencia del Accuracy a medida que aumenta en número de modelos base, siendo el óptimo 250 modelos bases.

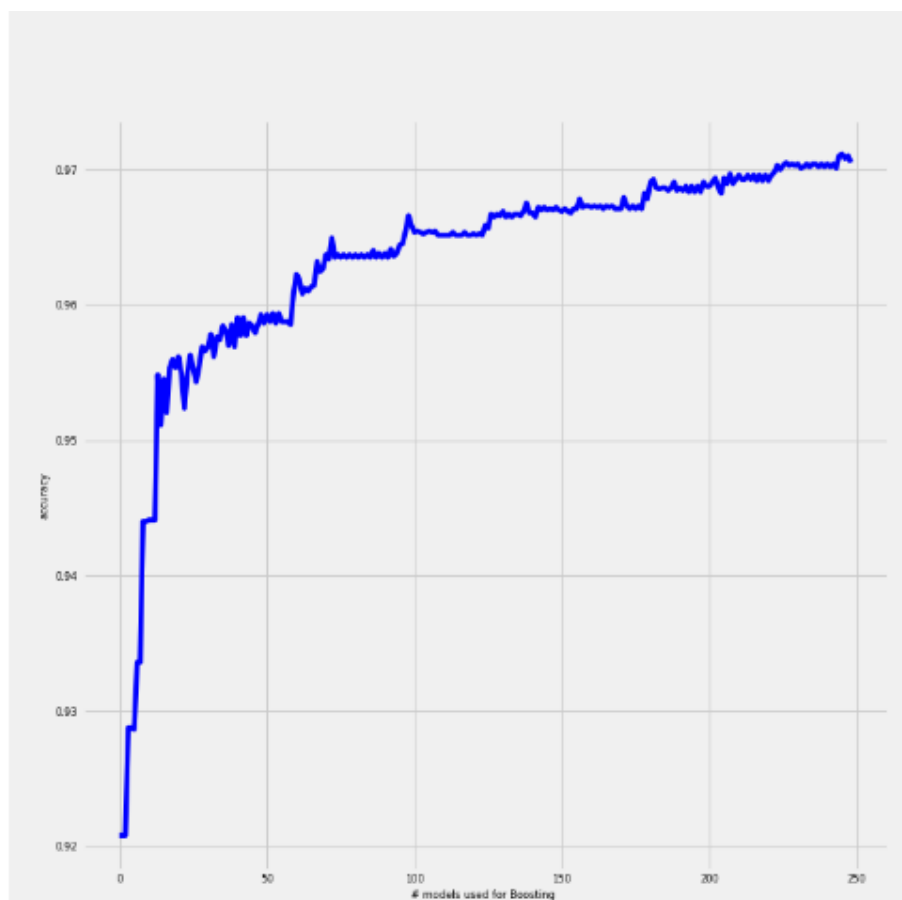


Figura 5.2: Numero de modelos bases

Cuadro 5.3: Comparación Bagging y Boosting con enfoque Batch ALERCE

	Accuracy	Precision	Recall	F1-score	Tiempo (Seg)
Naive Bayes	91,37 %	94,63 %	91,37 %	92,47 %	0,3338
Bagging	91,68 %	94,66 %	91,68 %	92,70 %	0,3570
Boosting	97,67	97,69	94,52 %	95,44 %	0,0080

5.1.3. Modelos Online:

A continuación se compararan las métricas de clasificación para los modelos Bagging y Boosting Online. Donde se observa que el rendimiento del modelo Boosting es el mejor, pero el que utilizo la mayor cantidad de tiempo.

Bagging Online

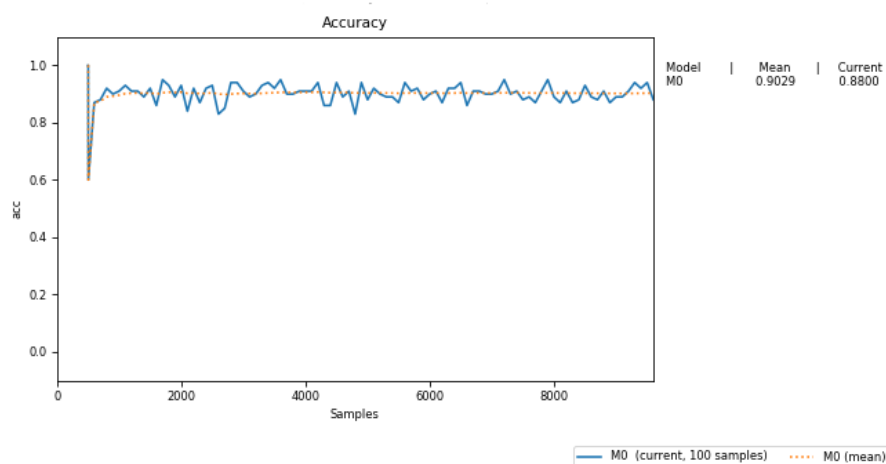


Figura 5.3: Bagging Online

Boosting Online

Dado el Accuracy y el tiempo de ejecución del modelo, el modelo Bagging es mucho mas rápido que el Boosting, siendo el Accuracy muy similar entre ambos.

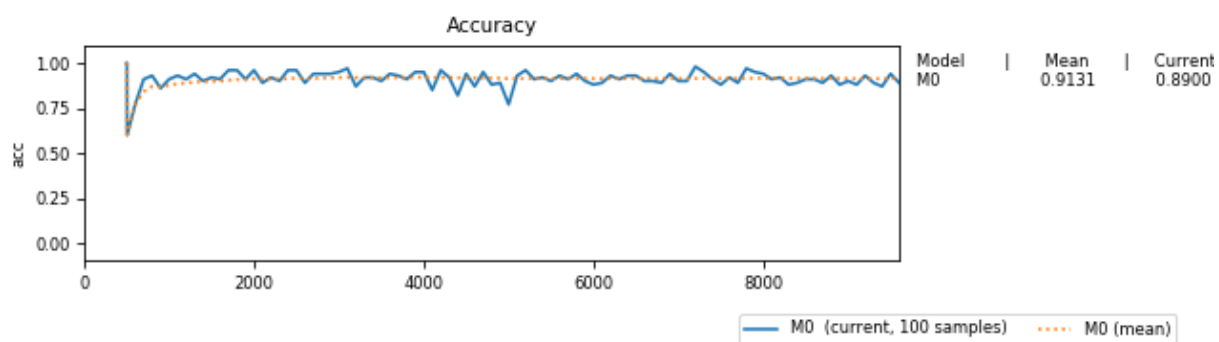


Figura 5.4: Boosting Online

Cuadro 5.4: Comparación Batch y Online ALeRCE

	Acurracy	Recall	Tiempo Online (Seg)	Tiempo Batch (seg)
Bagging	90,26 %	91,34 %	1258	926
Boosting	91,31 %	91,79 %	2531	1607

Capítulo 6

Conclusiones

Los modelos de Ensemble Learning han tomado protagonismo en los últimos años ya que al usar varios modelos para abordar el mismo problema, reducen de forma significativa los errores que comete un solo Weak Learner, dándole no solo un mayor poder predictivo al momento de clasificar, sino que también reduce la posibilidad de sobre-ajuste de los modelos con los datos de entrenamiento lo que reduce su capacidad para generalización con nuevos datos. En esta investigación se comprobó como al usar modelos de Ensemble Learning mejora la precisión de los Weak Learners, comprobando la hipótesis de Breiman (1996) donde el uso de varios clasificadores débiles logran un clasificador débil. Respecto a esta hipótesis, los modelos de Ensemble Learning fueron probados y entrenados con datos mediante un enfoque Batch o lote, lo cual es útil cuando se dispone de los recursos computacionales para el correcto procesamiento de información.

El principal objetivo de esta investigación era probar los modelos de Ensemble Learning dando un enfoque Online o streaming, ya que en la actualidad, en gran cantidad de problemas los datos tienen un gran volumen de información, lo cual hace costoso el entrenamiento de los modelos. Planteando el uso de los modelos de Ensemble Learning online para procesar un flujo de datos que llegan constantemente, pre-entrenando los modelos con una cantidad acotada de información se probó que aquellos modelos utilizan menos

recursos al ser entrenados previamente con un conjunto acotado el cual posteriormente se re-entrena con los mini-batch de llegada en el flujo, el cual reduce el tiempo de procesamiento obteniendo resultados similares a sus alternativas con enfoque en batch. Tanto el modelo Bagging y Boosting introducido por N.Oza en Oza (2005), tienen rendimientos similares, los cuales al optimizar los parámetros como el numero de modelos base convergen a sus alternativas Batch. Comparando los modelos de Ensemble y usando la parsimonia con la relación entre precisión y tiempo de ejecución, el Bagging Online es la mejor opción al clasificar, ya que los tiempos son mas razonables comparando con Boosting. Los modelos con enfoque online, son una buena alternativa a problemas donde la cantidad de información a procesar es demasiado grande y aumentan de forma constante, como lo son datos astronómicos, sensores, los cambios en la bolsa y juegos online que generan datos constantemente y existe la necesidad de procesarlos de forma adecuada.

Dada la naturaleza del lenguaje Python, que fue la herramienta en la que se desarrollo gran parte de este trabajo, se abren muchas oportunidades de incorporar estos modelos de procesamiento online, tanto en aplicaciones web, Inteligencia artificial o Inteligencia de negocios, donde día a día los software open source han tomado un gran protagonismo, ya que cada vez, la comunidad de usuario de estos software crecen, las cuales aportan contenidos y aplicaciones a problemas cada vez mas complejos, lo que ayuda a desarrollar tecnología cada vez mas avanzada y autónoma, sin la necesidad de disponer de licencias costosas. También Python y R han sido incorporados en los servicios de programación en la nube para procesamiento masivo de información como Amazon Web Service, Azure y databricks, como también en software de inteligencia de negocios como Power BI, incorporando todo el arsenal para visualizar información de manera interactiva.

Capítulo 7

Anexos

```
##### Script datos simulados #####  
#Importar librerias ##  
%matplotlib notebook  
import warnings  
warnings.filterwarnings('ignore')  
import pandas as pd  
import numpy as np  
import matplotlib.pyplot as plt  
import seaborn as sns  
from skmultiflow.data import WaveformGenerator  
from skmultiflow.trees import HoeffdingTree  
from skmultiflow.evaluation import EvaluatePrequential  
import matplotlib.pyplot as plt  
from sklearn.naive_bayes import GaussianNB  
from skmultiflow.meta import OzaBagging  
from skmultiflow.evaluation import EvaluatePrequential  
from skmultiflow.data.file_stream import FileStream
```

```
## Simular un stream dataset

# 1. Crear el stream

stream = WaveformGenerator()

stream.prepare_for_use()

# 2 Bagging Oza con base naives bayes

nb=GaussianNB()

clf = OzaBagging(base_estimator=nb, n_estimators=10)

clf

## Evaluacion Oza bagging con Naives bayes como Weak Learners

# 3. Configurar el evaluador

evaluator = EvaluatePrequential(show_plot=True,

                                pretrain_size=500,

                                max_samples=40000,metrics=['accuracy', 'recall'])

# 4. Run evaluation

evaluator.evaluate(stream=stream, model=clf)

from skmultiflow.meta import OnlineBoosting

from skmultiflow.bayes import NaiveBayes

    # Boosting Oza con base naives bayes

nb=GaussianNB()

clf2 = OnlineBoosting(base_estimator=NaiveBayes(), n_estimators=10)

clf2

# 5. Configurar el evaluador

evaluator = EvaluatePrequential(show_plot=True,

                                pretrain_size=500,

                                max_samples=20000)

# 6. Run evaluation
```

```
evaluator.evaluate(stream=stream, model=clf2)

## Entrenamiento de modelos ##

from sklearn.ensemble import BaggingClassifier
from sklearn.datasets import make_classification

#Separar datos

from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test, = train_test_split(X, y, test_size = 0.3,
random_state = 63)

# Weak Learners:

##### Logistica Simple #####

from sklearn import feature_selection
from sklearn import linear_model
model = linear_model.LogisticRegression()
model.fit(X_train,y_train)
predictions = model.predict(X_test)
model.score(X_test,y_test)

from sklearn.metrics import f1_score
from sklearn.metrics import precision_recall_fscore_support
precision_recall_fscore_support(y_test, predictions, average='weighted')
f1_score(y_test, predictions, zero_division=1)

##### Arbol #####

from sklearn import tree
model_arbol = tree.DecisionTreeClassifier()
model_arbol.fit(X_train,y_train)
predictions = model_arbol.predict(X_test)
model_arbol.score(X_test,y_test)
```

```
precision_recall_fscore_support(y_test, predictions, average='weighted')
f1_score(y_test, predictions, zero_division=1)
```

```
##### Naive Bayes Classifier #####
```

```
from sklearn.naive_bayes import GaussianNB
model_nbayes = GaussianNB()
model_nbayes.fit(X_train, y_train)
predictions = model_nbayes.predict(X_test)
model_nbayes.score(X_test, y_test)
nb_prediction_proba = model_nbayes.predict_proba(X_test)[:, 1]
model_nbayes.score(X_test, y_test)
precision_recall_fscore_support(y_test, predictions, average='weighted')
f1_score(y_test, predictions, zero_division=1)
```

```
##### Perceptron #####
```

```
from sklearn.metrics import f1_score
from sklearn.datasets import load_digits
from sklearn.linear_model import Perceptron
percep = Perceptron(tol=1e-3, random_state=0)
percep.fit(X_train, y_train)
predictions = percep.predict(X_test)
percep.score(X_test, y_test)
rf_prediction_proba = percep.predict(X_test)
percep.f1_score(X_test, y_test)
precision_recall_fscore_support(y_test, predictions, average='weighted')
f1_score(y_test, predictions, zero_division=1)
```

```
### Bagging Naives Bayes
```

```
model_bag3 = BaggingClassifier(model_nbayes,  
                               n_estimators=100,  
                               bootstrap = True, random_state = 1)  
model_bag3.fit(X_train,y_train)  
predictions = model_bag3.predict(X_test)  
model_bag3.score(X_test,y_test)  
precision_recall_fscore_support(y_test, predictions, average='weighted')  
f1_score(y_test, predictions, zero_division=1)
```

```
#### Boosting Naives Bayes
```

```
from sklearn.ensemble import AdaBoostClassifier  
model_boos = AdaBoostClassifier(base_estimator=model_nbayes, n_estimators=250,  
                                learning_rate=1.5, random_state=1)  
  
model_boos.fit(X_train,y_train)  
predictions = model_boos.predict(X_test)  
model_boos.score(X_test,y_test)  
precision_recall_fscore_support(y_test, predictions, average='weighted')  
f1_score(y_test, predictions, zero_division=1)
```

```
##### Online Ensemble Learning con datos desde un CSV #####
```

```
stream_1 = FileStream('C:/Users/yerko/Anaconda3/pkgs/scikit-multiflow-0.4.1-py37h79cbd  
Lib/site-packages/skmultiflow/data/datasets/df_fin_ALE_1.csv', target_idx=-1)  
stream_1
```

```
stream_1.prepare_for_use()
# Bagging Oza con base naives bayes
nb=GaussianNB()
clf = OzaBagging(base_estimator=nb, n_estimators=10)
clf
# 3. Configurar evaluador
clf = OzaBagging(base_estimator=nb, n_estimators=10)
evaluator = EvaluatePrequential(show_plot=True,
                                pretrain_size=1000, n_wait = 100,
                                max_samples=10000, metrics=['accuracy','recall','f1'])
# 4. Run evaluation
evaluator.evaluate(stream=stream, model=clf)

### Boosting Online con Naive Bayes como weak learner:
from skmultiflow.meta import OnlineBoosting
# Boosting Oza con base naives bayes
nb1=NaiveBayes()
clf2 = OnlineBoosting(base_estimator=nb1, n_estimators=10)
clf2

# 3. Configurar evaluador
evaluator = EvaluatePrequential(show_plot=True,
                                pretrain_size=1000,n_wait = 100,
                                max_samples=10000, metrics=['accuracy','recall','f1'])
# 4. Run evaluation
evaluator.evaluate(stream=stream, model=clf2)
```

Referencias

- Barandiaran, I. (1998). The random subspace method for constructing decision forests. *IEEE Trans. Pattern Anal. Mach. Intell*, 20(8):1–22.
- Battista, D., Cristaldo, T., Ramos, L., Nuñez, J., Retamar, S., Herrera, D. B. N., Hazan, E., Kale, S., and Luo, H. (2016). Minería de datos aplicada a datos masivos.
- Bellm, E. C., Kulkarni, S. R., Graham, M. J., Dekany, R., Smith, R. M., Riddle, R., Masci, F., Helou, G., Prince, T. A., Adams, S. M., Barbarino, C., Barlow, T., Bauer, J., Beck, R., Belicki, J., Biswas, R., Blagorodnova, N., Bodewits, D., Bolin, B. T., Brinnel, V., Brooke, T., Bue, B. D., Bulla, M., Burruss, R. S., Cenko, S. B., Chang, C., Connolly, A., Coughlin, M. W., Cromer, J., Cunningham, V. L., De, K., Delacroix, A., Desai, V., Duev, D. A., Eadie, G. M., Farnham, T., Feeney, M. E., Feindt, U., Flynn, D., Franckowiak, A., Frederick, S., Fremling, C., Gal-yam, A., Gezari, S., Giomi, M., Goldstein, D. A., Golkhou, V. Z., Goobar, A., Groom, S., Hacopians, E., Hale, D., Henning, J. A., Ho, A. Y. Q., Hover, D., Howell, J. H., Hung, T., Huppenkothen, D., Imel, D., Ip, W.-H., vZeljko Ivezi’c, Jackson, E., Jones, L., Juric, M., Kasliwal, M. M., Kaspi, S., Kaye, S. B., Kelley, M. S., Kowalski, M., Kramer, E., Kupfer, T., Landry, W., Laher, R. R., Lee, C. D., Lin, H. W., Lin, Z., Lunnan, R., Mahabal, A., Mao, P. H., Miller, A. A., Monkevitz, S. M., Murphy, P., Ngeow, C.-C., Nordin, J., Nugent, P., Ofek, E., Patterson, M. T., Penprase, B. E., Porter, M., Rauch, L., Rebbapragada, U., Reiley, D. J., Rigault, M., Rodríguez, H., van Roestel, J., Rusholme, B., van Santen, J., Schulze, S., Shupe, D. L., Singer,

- L. P., Soumagnac, M. T., Stein, R., Surace, J., Sollerman, J., Szkody, P., Taddia, F., Terek, S., van Sistine, A., van Velzen, S., Vestrand, W. T., Walters, R., Ward, C., Ye, Q., Yu, P.-C., Yan, L., and Zolkower, J. (2019). The zwicky transient facility: System overview, performance, and first results. *Publications of the Astronomical Society of the Pacific*, 131:018002–018002.
- Beygelzimer, A., Hazan, E., Kale, S., and Luo, H. (2015). Online gradient boosting.
- Breiman, L. (1996). Bagging predictors. *Machine learning*, 24(2):123–140.
- Breiman, L. (1999). Pasting small votes for classification in large databases and on-line. *Machine learning*, 36(1-2):85–103.
- Chipman, H. A., George, E. I., and McCulloch, R. E. (1998). Bayesian cart model search. *Journal of the American Statistical Association*, 93(443):935–948.
- Chivers, I. and Sleightholme, J. (2015). An introduction to algorithms and the big o notation. In *Introduction to Programming with Fortran*, pages 359–364. Springer.
- Cunha, T. D. S. (2011). General id3 problem solver.
- Dietterich, T. and Bakiri, T. (1994). Solving multiclass learning problems via error-correcting output codes. *Journal of artificial intelligence research*.
- Eyheramendy, S., Elorrieta, F., and Palma, W. (2018). An irregular discrete time series model to identify residuals with autocorrelation in astronomical light curves. *Monthly Notices of the Royal Astronomical Society*, 481(4):4311–4322.
- Ferrer, F. J., Aguilar, J. S., and Riquelme, J. C. (2005). Aprendizaje incremental de reglas en data streams. *Actas del III Taller Nacional de Minería de Datos y Aprendizaje, TAMIDA2005*, pages 261–270.

- Freund, Y. and Schapire, R. (1996). A decision-theoretic generalization of on-line learning and an application to boosting, department of computer science and technology, university of peloponnese, greece.
- Freund, Y., Schapire, R., and Abe, N. (1999). A short introduction to boosting. *Journal-Japanese Society For Artificial Intelligence*, 14(771-780):1612.
- Hosmer Jr, D. W., Lemeshow, S., and Sturdivant, R. X. (2013). *Applied logistic regression*, volume 398. John Wiley & Sons.
- Kotsiantis, S. B. (2007). Supervised machine learning a review of classification techniques.
- Kuncheva, L. I. (2008). Classifier ensembles for detecting concept change in streaming data: Overview and perspectives. In *2nd Workshop SUEMA*, volume 2008, pages 5–10.
- Louppe, G. and Geurts, P. (2012). Ensembles on random patches. In *Joint European Conference on Machine Learning and Knowledge Discovery in Databases*, pages 346–361. Springer.
- Lyon, R. J., Stappers, B. W., Cooper, S., Brooke, J. M., and Knowles, J. D. (2015). Fifty years of pulsar candidate selection: From simple filters to a new principled real-time classification approach, monthly notices of the royal astronomical society. (1104-1123).
- Montiel, J., Read, J., Bifet, A., and Abdessalem, T. (2018). Scikit-multiflow: A multi-output streaming framework. *Journal of Machine Learning Research*, 19(72):1–5.
- Nun, I., Protopapas, P., Sim, B., Zhu, M., Dave, R., Castro, N., and Pichara, K. (2015). Fats: Feature analysis for time series.
- Oza, N. (2000). Online ensemble learning. *A dissertation submitted in partial satisfaction of the requirements for the degree of Doctor of Philosophy in Computer Science*, 1(13):15–17.

- Oza, N. C. (2005). Online bagging and boosting. In *2005 IEEE international conference on systems, man and cybernetics*, volume 3, pages 2340–2345. Ieee.
- Patil, T. R., Sherekar, S., et al. (2013). Performance analysis of naive bayes and j48 classification algorithm for data classification. *International journal of computer science and applications*, 6(2):256–261.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., and Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830.
- Python Software Foundation (1995). *Python Language Reference, versión 2.7*. Amsterdam, Netherlands.
- R Core Team (2014). *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria.
- R Core Team (2019). *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria.
- Rocca, J. (2019). Ensemble methods: bagging, boosting and stacking. <https://towardsdatascience.com/ensemble-methods-bagging-boosting-and-stacking-c9214a10a205>.
- Russo, C., Ramón, H. D., Alonso, N., Cicerchia, L. B., Esnaola, L., and Tessore, J. P. (2016). Tratamiento masivo de datos utilizando técnicas de machine learning. In *XVIII Workshop de Investigadores en Ciencias de la Computación (Entre Ríos, Argentina)*.
- SONG, T. (2018). Implementing the perceptron algorithm from scratch with python. <https://tianyusong.com/2018/02/23/implementing-the-perceptron-algorithm-from-scratch-with-python/>.

-
- Stephen, I. (1990). Perceptron-based learning algorithms. *IEEE Transactions on neural networks*, 50(2):179.
- Tsang, I. W., Kwok, J. T., and Cheung, P.-M. (2005). Core vector machines: Fast svm training on very large data sets. *Journal of Machine Learning Research*, 6(Apr):363–392.
- Tsangaratos, P. and Ilia, I. (2016). Comparison of a logistic regression and naïve bayes classifier in landslide susceptibility assessments: The influence of models complexity and training dataset size. *Catena*, 145:164–179.
- Visa, S., Ramsay, B., Ralescu, A. L., and Van Der Knaap, E. (2011). Confusion matrix-based feature selection. *MAICS*, 710:120–127.
- Wijffels, J. (2014). RMOA: Connect r with moa to perform streaming classifications. R package version 1.0.